جامعة فيلادلفيا

Philadelphia University

Project Title: Access Tracing

Submitted by:

Tamer Thabet

Department of Cyber Security

Faculty of Information Technology

Philadelphia University – Jordan

Approval Page

This project entitled "Access Tracing" has been prepared and submitted by the following students in partial fulfillment of the requirements for the bachelor's degree in Cybersecurity / Information Security.

This project has been reviewed and approved by the assigned supervisor and the examination committee.

Supervisor: Dr. Athari Al-Natsheh

Signature: _____

Date: _____

Committee Member:
Name: _____

Signature: _____

Date: _____

Committee Member:
Name: _____

Signature: _____

Date: _____

Committee Member:
Name: _____

Signature: _____

Date: _____

Certificate

This is to certify that the graduation project titled **"Access Tracing"** was carried out by:

- Tamer Thabet
- Omar Al-Azzam
- Hussien Adel
- Amer Abo-Najim

under the supervision of **Dr. Athari Al-Natsheh**, in partial fulfillment of the requirements for the bachelor's degree in Cybersecurity / Information Security at Philadelphia University.

The project has been completed according to the academic regulations and standards of the university.

Supervisor Signature: _____

Date: _____

Dedication

We dedicate this work to our families, whose continuous support, encouragement, and patience made this achievement possible.

We also dedicate it to our instructors and mentors who guided us throughout our academic journey and inspired our interest in cybersecurity and information security.

Acknowledgement

We thank our families and friends for their encouragement and motivation during all stages of this academic endeavor.

AND we would like to express our sincere gratitude to our supervisor, **Dr. Athari Al-Natsheh**, for her guidance, valuable feedback, and continuous support throughout the development of this project. Her expertise and direction played a crucial role in shaping the structure and quality of this work.

Contents

Project Title: Access Tracing I
Approval Page II
Certificate III
Dedication IV
Acknowledgement V
Contents VI
Tables IX
List of Figures X
Abstract 1
CHAPTER 1 – INTRODUCTION 2
1.1 Introduction 2
1.2 Background 2
1.3 Problem Statement 3
1.4 Limitations 3
1.5 Project Objectives 4
1.6 Project Solution Overview 4
1.7 Project Scope 4
1.8 Project Feasibility 5
1.9 Project Plan 5
CHAPTER 2 – LITERATURE REVIEW 7
2.1 Introduction 7
2.2 Review of Existing Systems 7
2.3 Comparison of Existing Solutions 9
Summary 11
CHAPTER 3 – METHODOLOGY AND WORK PLAN 12
3.1 Introduction 12
3.2 System Development Methodology (Waterfall Model) 13
3.2.1 Requirement Analysis 14
3.2.2 System Design 14
3.2.3 Implementation 14

3.2.4 Testing 15
3.2.5 Deployment and Documentation 15
3.3 Requirement Gathering Techniques 15
3.4 Project Plan 16
3.5 Development of Tools and Technologies 17
CHAPTER 4 – PROJECT DESIGN AND SPECIFICATION 19
4.1 Introduction 19
4.2 System Specification 20
4.2.1 Functional Requirements 20
4.2.2 Non-Functional Requirements 20
4.3 Database Design 21
4.4 User Interface Design 21
4.5 Chapter Summary 21
CHAPTER 5 – IMPLEMENTATION 22
5.1 Introduction 22
5.2 Virtual Environment Setup 22
5.3 Snort IDS Installation and Configuration 25
5.4 Windows Server Auditing and Web Application Deployment 27
5.5 SQL Injection Attack Execution 29
5.6 Cross-Site Scripting (XSS) Attack Execution 30
5.7 Log Ingestion and Data Normalization 32
5.8 Database Storage Implementation 34
5.9 Dashboard Implementation and Visualization 34
5.10 Chapter Summary 36
CHAPTER 6 – SYSTEM IMPLEMENTATION 37
6.1 Introduction 37
6.2 Implementation Overview 38
6.3 Implementation Details 38
6.3.1 User Authentication 38
6.3.2 Database Integration 39
6.3.3 User Interface Elements 40

6.3.4 Functionality Modules 40
CHAPTER 7 – RESULTS AND DISCUSSION 43
7.1 Introduction 43
7.2 Data Collection and Research 43
7.3 System Testing and User Feedback 43
CHAPTER 8 – RECOMMENDATIONS AND CONCLUSION 45
8.1 Introduction 45
8.2 Conclusion 45
8.3 Limitations and Recommendations for Future Work 45
REFERENCES 47
APPENDICES: 48

Tables

Table 2.1: Comparison of Existing Monitoring and Correlation Solutions_____ 21

List of Figures

- Fig 1.1: Project Plan _____ 6
- Fig 3.1: Access Tracing System Architecture _____ 12
- Fig 3.2: Waterfall Development Methodology Applied to Access Tracing _____ 13
- Fig 3.3: Testing Workflow for Access Tracing _____ 16
- Fig 3.4: Access Tracing Development Timeline _____ 17
- Fig 4.1: Access Tracing End-To-End Architecture _____ 19
- Fig 5.1.1: Virtual Laboratory Network Configuration _____ 22
- Fig 5.1.2: Virtual Laboratory Network Configuration _____ 23
- Fig 5.1.3: Virtual Laboratory Network Configuration _____ 23
- Fig 5.2.1: IP Address Configuration for Kali Linux & Window Server _____ 24
- Fig 5.2.2: IP Address Configuration for Kali Linux & Window Server _____ 24
- Fig 5.3.1: Snort Configuration and Custom Rule Setup _____ 25
- Fig 5.3.2: Snort Configuration and Custom Rule Setup _____ 25
- Fig 5.3.3: Snort IDS main configuration file _____ 26
- Fig 5.3.4:Snort Rules _____ 26
- Fig 5.3.5: Snort Alert Log Monitoring _____ 27
- Fig 5.3.6: Snort Alert Triggered by SQL Injection _____ 27
- Fig 5.3.7: Snort Alert Triggered by XSS Script _____ 27
- Fig 5.4.1: Windows Server Advanced Audit Policy Configuration _____ 28
- Fig 5.4.2: Windows Security Authentication Events _____ 28
- Fig 5.4.3: Windows Security Authentication Events _____ 29
- Fig 5.5.1: Normal Web Application Response _____ 29
- Fig 5.5.2: SQL Injection Payload Submission _____ 30
- Fig 5.5.3: Application Response After SQL Injection _____ 30
- Fig 5.5.4: SQL Injection Using UNION SELECT _____ 30
- Fig 5.6.1: Reflected XSS Payload Injection _____ 31

- Fig 5.6.2: XSS Script Execution Evidence _____ 31
- Fig 5.6.3: XSS-Based Page Manipulation _____ 32
- Fig 5.6.4: XSS Filter Bypass Using HTML Event Handler _____ 32
- Fig 5.7.1: PowerShell Log Ingestion Script Execution _____ 33
- Fig 5.7.2: Retrieved Snort Alert Log _____ 33
- Fig 5.8: Stored Snort Alerts in MySQL Database _____ 34
- Fig 5.9.1: Dashboard Summary of Intrusion Activity _____ 35
- Fig 5.9.2: Dashboard View of User Authentication Events _____ 35
- Fig 5.9.3: Administrative Dashboard Displaying Correlated Alerts _____ 36
- Fig 6.1: Test Scenario Execution Workflow _____ 37
- Fig 6.2: Administrative Authentication Interface _____ 39
- Fig 6.3: MySQL Database Records After Integration _____ 39
- Fig 6.4: Administrative Dashboard User Interface _____ 40
- Fig 6.5.1: Attack Simulation _____ 41
- Fig 6.5.2: Snort IDS Mode Detects Traffic _____ 41
- Fig 6.5.3: Alert Written to Log File _____ 41
- Fig 6.5.4: PowerShell Ingestion and Data Inserted to MySQL _____ 42
- Fig 6.5.5: Dashboard Displays _____ 42

Abstract

Access Tracing is a unified monitoring platform designed to reconstruct and correlate access activities across network and host layers. The system collects intrusion detection alerts and operating system authentication events, normalizes them into a centralized data store, and presents them through an administrative dashboard for analysis and review.

The platform focuses on tracing how access attempts originate, how they propagate through the network, and how they interact with system services. By combining network-level detection with host-level auditing, Access Tracing provides visibility into attack behavior such as probing, scanning, and web-based exploitation attempts.

The system follows a structured development lifecycle and is implemented using lightweight, widely adopted technologies. Its architecture emphasizes clarity, traceability, and operational reliability, making it suitable for controlled environments that require clear insight into access behavior and security events.

CHAPTER 1 – INTRODUCTION

1.1 Introduction

In modern information systems, access to digital resources occurs continuously through a wide range of interactions including authentication attempts, network requests, service calls, and application-level inputs. Each of these interactions generates security-relevant evidence that reflects how users and systems interact with protected resources. As systems grow more interconnected and services become increasingly exposed to external networks, the complexity of monitoring access behavior rises significantly. Cyber-attacks no longer rely on a single action; instead, they unfold through sequences of events that span multiple layers of the system.

Attackers typically begin with reconnaissance activities, followed by probing services, exploiting vulnerabilities in applications, and attempting unauthorized authentication. These stages are distributed across the network and the host, making it difficult to understand the full attack path without centralized visibility. When security events are captured in isolation, analysts lose the ability to trace how a network event leads to host-level consequences. This lack of correlation creates blind spots that attackers exploit.

Access Tracing addresses this challenge by focusing on the reconstruction of access behavior across system layers. The concept of Access Tracing refers to collecting, correlating, and analyzing events that describe who attempted to access the system, how the access was attempted, from where it originated, and what impact it produced on the host environment. Rather than treating network alerts and host logs as separate artifacts, Access Tracing unifies them into a single analytical alert.

This project presents the design and implementation of an Access Tracing system that integrates network intrusion detection alerts generated by Snort with authentication and application-related logs collected from a Windows Server environment. The system provides centralized visibility through a web-based dashboard that enables administrators to monitor, analyze, and trace access attempts in a structured and readable manner. The solution is implemented in a controlled virtual environment and focuses on realistic attack scenarios, including SQL Injection and Cross-Site Scripting attacks, to demonstrate traceability from attack execution to recorded evidence.

1.2 Background

Cybersecurity monitoring has traditionally relied on separate tools to observe different layers of system activity. Network Intrusion Detection Systems (NIDS), such as Snort, analyze network traffic and identify suspicious patterns, protocol misuse, and malicious payloads. These systems are effective at detecting reconnaissance scans, malformed packets, and application-layer attacks transmitted over the network. However, they provide limited insight into what occurs after traffic reaches the target host.

On the other hand, host-based logging mechanisms, particularly Windows Event Viewer, record detailed information about authentication attempts, account activity, privilege usage, and system interactions. These logs provide deep visibility into user behavior but lack context about the network activity that triggered them. As a result, analysts are forced to manually correlate network alerts with host logs, a time-consuming and error-prone process.

Enterprise environments often address this limitation using Security Information and Event Management (SIEM) systems. SIEM platforms ingest logs from multiple sources, normalize data, correlate events, and present dashboards for monitoring and investigation. Despite their effectiveness, SIEM solutions are complex, resource-intensive, and costly. Their deployment requires advanced infrastructure, distributed collectors, and licensing models that are not suitable for educational environments or small-scale laboratories.

This gap creates the need for a lightweight system that demonstrates core access tracing and correlation concepts without enterprise overhead. The Access Tracing project fills this gap by providing a simplified yet functional system that bridges network-level detection and host-level logging into a unified dashboard. The system emphasizes clarity, traceability, and practical understanding of access monitoring principles.

1.3 Problem Statement

Although modern systems generate extensive security logs, these logs are fragmented across different platforms and tools. Network intrusion alerts, authentication records, and application-level events are stored separately, preventing effective correlation. Without unified visibility, it is difficult to determine whether a detected network attack resulted in a successful host-level action or remained an unsuccessful attempt.

This fragmentation leads to several operational problems, including delayed detection of attacks, increased false positives, difficulty in reconstructing attack timelines, and limited forensic investigation capabilities. Analysts may observe a SQL Injection alert in the IDS without knowing whether the payload affected the application or database. Similarly, multiple failed login attempts may appear in host logs without clear linkage to the originating network activity.

The core problem addressed by this project is the absence of a unified system that traces access attempts across both the network and host layers. There is a need for a monitoring solution that correlates Snort IDS alerts with Windows authentication and application logs in near real time, enabling administrators to trace access behavior holistically and accurately.

1.4 Limitations

The Access Tracing system is developed within a controlled virtual environment and under academic time constraints. The system operates using a limited number of virtual machines, including a single Windows Server instance and a single Kali Linux machine running Snort. This limits scalability and prevents simulation of large enterprise traffic volumes.

The project relies on controlled and simulated attack scenarios rather than real organizational datasets. As a result, the diversity of attack patterns is limited to predefined scenarios such as reconnaissance, SQL Injection, and XSS. Additionally, the system focuses on detection, correlation, and visualization rather than automated response or advanced behavioral analytics. Despite these limitations, the system fulfills its goal of demonstrating effective access tracing in a practical and controlled environment.

1.5 Project Objectives

The primary objective of the Access Tracing project is to design and implement a unified monitoring system capable of tracing access attempts across both network and host environments in a clear and structured manner. The system aims to capture network-based attack indicators through an intrusion detection mechanism while simultaneously recording host-level authentication and application-related activities. By correlating these events, the project seeks to provide a comprehensive view of how access attempts originate, propagate, and affect the target system. The project also aims to demonstrate practical monitoring techniques through realistic attack scenarios, including SQL Injection and Cross-Site Scripting, enabling traceability from attack execution to recorded evidence. Ultimately, the objective is to deliver a functional and practical access tracing platform that supports monitoring, analysis, and incident investigation within a controlled environment.

1.6 Project Solution Overview

The Access Tracing project proposes a multi-layered solution that integrates network-level detection with host-level logging into a centralized monitoring platform. The solution relies on Snort IDS to observe network traffic and generate alerts when suspicious or malicious activity is detected. In parallel, Windows Server records authentication attempts and application-related events that reflect user behavior and system interaction. Automated ingestion scripts retrieve these logs at regular intervals, normalize their formats, and store them in a structured database designed to support efficient querying and analysis. A web-based administrative dashboard retrieves the stored data through dedicated interfaces and presents it in an organized format that enables administrators to trace access attempts across system layers. This approach provides continuous visibility into access behavior while maintaining a lightweight and deployable architecture suitable for a virtual laboratory environment.

1.7 Project Scope

The scope of the Access Tracing project encompasses all functional components required to implement a unified access monitoring and tracing system within a controlled virtual environment. This includes the deployment and configuration of Snort IDS for network traffic monitoring, the activation of Windows Server auditing policies to capture authentication and application-level events, the development of automated scripts for log extraction and ingestion, and the design of a centralized database for structured event storage. The project also covers the implementation of a web-based dashboard that enables real-time visualization and analysis of

correlated events. Furthermore, the scope includes the execution of controlled attack scenarios, such as SQL Injection and Cross-Site Scripting, to validate system functionality and demonstrate traceability. Elements such as enterprise-scale deployment, automated response mechanisms, advanced analytics, and large-scale data processing are intentionally excluded to maintain focus, feasibility, and clarity within the project's defined boundaries.

1.8 Project Feasibility

The feasibility of the Access Tracing project has been evaluated from technical, operational, and economic perspectives to ensure its practicality within the available timeframe and resources. From a technical standpoint, the project is achievable using widely available and open-source tools such as Snort, Windows Server auditing features, MySQL, PHP, and PowerShell, all of which can be deployed within a virtualized environment. The system architecture is designed to remain simple while maintaining functional completeness, reducing the risk of implementation complexity. Operational feasibility is supported by the system's relevance to access monitoring and security analysis, as well as the usability of the dashboard interface, which enables effective monitoring without requiring extensive training. Economic feasibility is ensured through the exclusive use of free software and minimal hardware requirements, making the project cost-effective while delivering meaningful monitoring and tracing capabilities.

1.9 Project Plan

The project plan for the Access Tracing system follows a structured and sequential approach to ensure organized development and timely completion. The planning phase began with requirement analysis, during which system objectives, log sources, attack scenarios, and monitoring needs were clearly defined. This was followed by the system design phase, where the overall architecture, data flow, database schema, and dashboard layout were established. After design approval, the implementation phase commenced, focusing on configuring Snort IDS, enabling Windows Server auditing, developing log ingestion scripts, and constructing the centralized database and dashboard interface. Once implementation was completed, the system underwent a dedicated testing phase in which controlled attack scenarios, including SQL Injection and Cross-Site Scripting, were executed to validate detection, correlation, and visualization accuracy. The final phase involved system stabilization, documentation preparation, and result analysis to ensure the system met its objectives. This structured plan ensured that each stage of development was completed systematically, reducing implementation risks and enabling clear progress tracking throughout the project lifecycle. The overall structure and timeline of the project are illustrated in Figure 1.9, which outlines the major phases from initiation to completion.

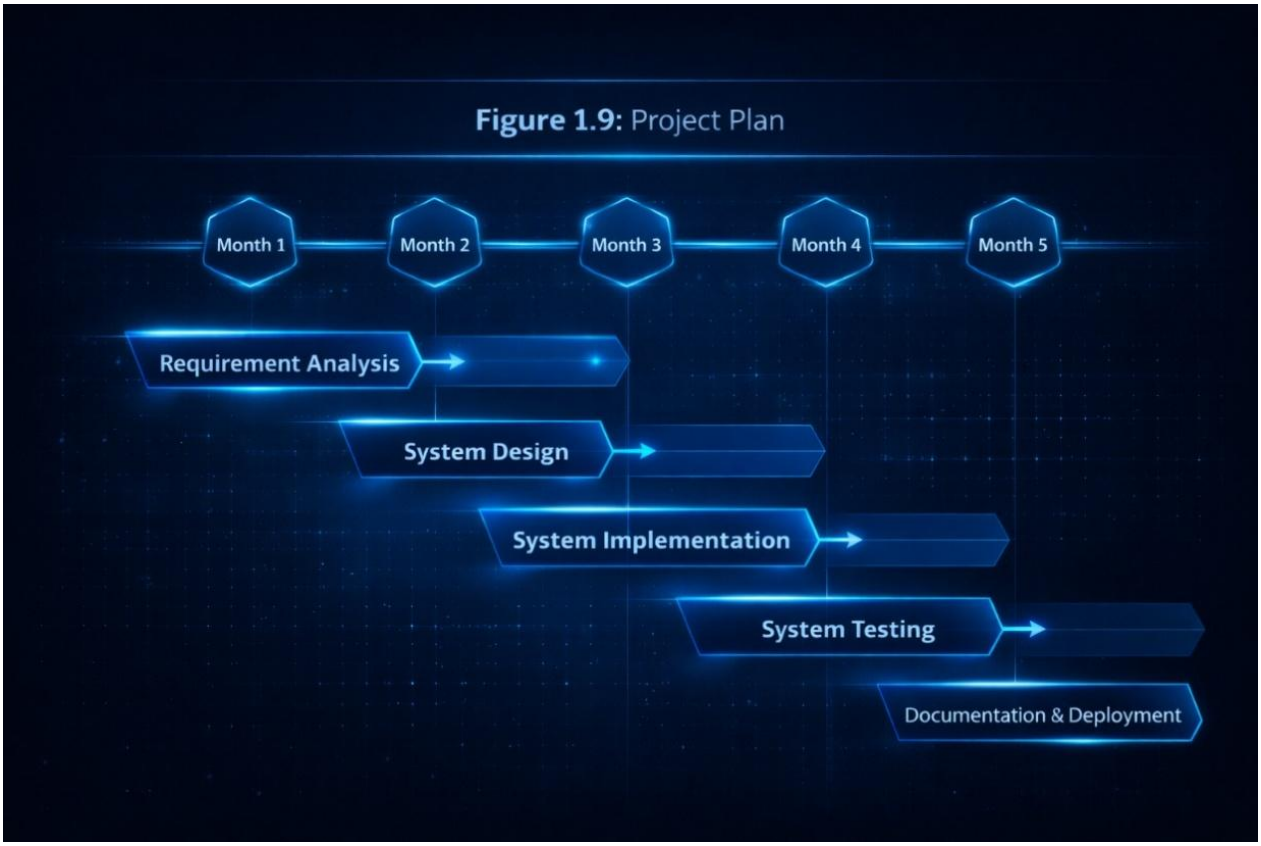

Fig 1.1: Project Plan

CHAPTER 2 – LITERATURE REVIEW

2.1 Introduction

A comprehensive literature review is essential for establishing the conceptual, technical, and practical foundations upon which the Access Tracing system is built. As cyber threats have evolved in complexity and scale, research in cybersecurity has increasingly emphasized the importance of achieving visibility across multiple layers of a system rather than relying on isolated monitoring mechanisms. Modern attacks are rarely confined to a single point of interaction; instead, they typically involve coordinated activities across the network layer, operating system, and application layer. This multi-stage nature of attacks has driven researchers to explore methods for correlating network telemetry with host-level events to improve detection accuracy and forensic analysis.

While a wide range of security tools exist, including network intrusion detection systems, host-based logging services, and centralized monitoring platforms, many of these solutions operate independently or are designed for large enterprise environments. As a result, they often lack the simplicity, accessibility, and flexibility required for educational settings or controlled laboratory deployments. This chapter reviews several key research efforts published between 2020 and 2024 that focus on log correlation, host–network integration, and multi-layer monitoring approaches. By analyzing how these systems were designed, implemented, and evaluated, the chapter identifies both strengths and limitations in existing solutions. These observations form the basis for motivating the design of the Access Tracing system as a lightweight and unified monitoring platform.

2.2 Review of Existing Systems

Steffen Haas et al. [1]

In this work, Haas, Sommer, and Fischer proposed a host–network correlation framework that integrated network telemetry generated by Zeek with host-level process information collected using Osquery. The primary objective of their system was to improve visibility into how network connections correspond to specific processes, users, and system activities on monitored endpoints. By combining flow metadata with detailed process tables, the framework enabled analysts to reconstruct chains of activity that linked external network communication to internal system behavior.

The architecture relied on distributed sensors deployed across Linux-based systems, where Zeek monitored network traffic and Osquery continuously queried host information. Correlation was achieved by aligning timestamps and identifiers across datasets, allowing analysts to trace network events to their originating processes. The study demonstrated that this approach significantly enhanced the detection of complex attack behaviors, particularly lateral movement and unauthorized process execution. However, the reliance on distributed Linux sensors, along

with the absence of a centralized visualization dashboard, increased deployment complexity and reduced suitability for small-scale or educational environments.

Sharma et al. [2]

In this work, Sharma and colleagues introduced a unified event analysis platform designed to aggregate logs from multiple sources, including Windows Event Logs, Linux system logs, and intrusion detection alerts. The system aimed to support forensic investigations by providing a centralized repository in which analysts could examine authentication activities, network anomalies, and system-level events together. Their approach emphasized log parsing, timestamp alignment, and normalization to enable consistent analysis across heterogeneous data sources.

The platform successfully demonstrated the value of centralized log aggregation for reconstructing incident timelines and identifying suspicious behavior patterns. However, the system primarily supported manual investigation workflows and lacked real-time visualization capabilities. Analysts were required to query and interpret logs directly, which limited the platform's effectiveness for continuous monitoring scenarios. Additionally, while the system unified multiple log sources, it did not emphasize direct correlation between network alerts and host authentication events in a unified visual context.

Albasheer et al. [3]

In this work, Albasheer and co-authors investigated the use of alert correlation techniques to enhance the effectiveness of network intrusion detection systems. Their research focused on reducing false positives by grouping related alerts and identifying attack sequences through similarity-based matching, rule-based correlation, and statistical modeling. The study highlighted the limitations of analyzing raw IDS alerts in isolation, emphasizing that alerts gain meaningful context only when processed through correlation engines.

The proposed system demonstrated improved detection accuracy by organizing alerts into structured sequences that reflected attacker behavior over time. Key contributions included the emphasis on normalization, temporal correlation, and contextual enrichment of alerts. Despite these strengths, the system focused exclusively on network-side data and did not incorporate host-level authentication or system logs. As a result, it lacked the ability to trace whether detected network attacks resulted in successful access or system-level impact.

Chen et al. [4]

In this work, Chen and colleagues developed a hybrid intrusion detection system that combined host-based features with network traffic attributes to improve detection accuracy. Their system employed data preprocessing, feature extraction, and a two-stage classification pipeline to analyze fused datasets. By leveraging datasets such as CICIDS2018 and UNSW-NB15, the authors demonstrated that combining host and network telemetry significantly outperformed traditional network-only intrusion detection systems.

The research showed strong improvements in detection rates and provided valuable insights into multi-layer monitoring. However, the system relied heavily on computationally intensive pipelines and complex processing frameworks. The dependence on large datasets and advanced processing made the solution unsuitable for lightweight or virtualized environments. Additionally, the system focuses on detection accuracy rather than operational traceability or real-time access monitoring.

Maosa et al. [5]

In this work, Maosa and collaborators proposed a hierarchical security event correlation model designed to link events in real time using similarity-based and graph-based techniques. Their framework aimed to reduce alert noise and reconstruct attack paths by organizing raw logs into structured correlation graphs. By analyzing relationships across multiple system layers, the model enhanced situational awareness and supported the identification of complex attack chains.

The study demonstrated promising results in correlating events across distributed environments and reconstructing attack timelines. However, the proposed framework assumed the availability of enterprise-grade logging pipelines and high-capacity processing infrastructure. The system did not specifically address the needs of small-scale deployments or educational environments, limiting its applicability in controlled laboratory settings.

2.3 Comparison of Existing Solutions

A comparative analysis of the reviewed studies reveals a common objective across all systems: improving security visibility by integrating or correlating data from multiple sources. The Zeek–Osquery framework achieved strong host–network linkage but required distributed Linux sensors and lacked centralized visualization. The unified event analysis platform centralized logs effectively but did not provide real-time correlation or monitoring capabilities. The alert correlation research enhanced IDS accuracy but focused solely on network alerts without host-level integration. The hybrid intrusion detection system demonstrated high detection accuracy through multi-layer fusion but required substantial computational resources. Finally, the hierarchical correlation model provided real-time event linking but targeted enterprise-scale deployments with heavy infrastructure requirements.

Collectively, these systems highlight the growing importance of unified monitoring and event correlation in modern cybersecurity environments. At the same time, they expose clear gaps related to deployment complexity, resource demands, lack of Windows-specific access tracing, and limited suitability for educational or virtualized environments. These gaps directly motivate the design of the Access Tracing system, which aims to provide a lightweight, VM-friendly platform capable of correlating Snort IDS alerts with Windows authentication logs in a clear and accessible dashboard.

Table 2.1: Comparison of Existing Monitoring and Correlation Solutions

Author / System	Year	Title	Primary Objective	Monitoring Layers	Correlation Method	Tools / Technologies	Datasets Used	ML Applied	Strengths	Limitations
Haas, Sommer & Fischer	2020	*Zeek-Oseary Host–Network Correlation Framework*	Correlation network flows with host processes	Host + Network	Flow ↔ Process correlation	Zeek, Osquery	None	No	Strong correlation; high visibility	Complex deployment; Linux-only; no dashboard
Sharma et al.	2021	*Unified Event Analysis Platform for Host and Network Monitoring*	Centralize logs from Windows, Linux, IDS for forensic analysis	Host + Network	Normalization + Timestamp alignment	Event Viewer, Syslog, Parsers	None	No	Multi-OS integration; strong forensic value	No real-time dashboard; weak correlation
Albasheer et al.	2022	*Cyber-Attack Prediction Using Alert Correlation Techniques in NIDS*	Reduce false positives via alert correlation	Network Only	Alert grouping + Similarity matching	NIDS logs, Correlation engine	Synthetic IDS alerts	Limited statistical models	Improved IDS accuracy; low noise	No host logs; no multi-layer visibility
Chen et al.	2023	*Hybrid Host–Network Intrusion Detection Using Feature Fusion*	Improve detection accuracy using ML and fused features	Host + Network	ML-based fusion + Classification	Python ML stack	CICIDS2018, UNSW-NB15	Yes	High accuracy; strong hybrid visibility	Requires high computational power
Maosa et al.	2024	*Hierarchical Security Event Correlation Model*	Real-time correlation using hierarchical graphs	Multi-layer (Host + Network)	Graph-based hierarchical correlation	SIEM logs, Graph engines	Enterprise logs	Partial	Real-time attack-path detection	Enterprise- scale only; heavy architecture
This Work	2025	*Unified Access Tracing System for Correlating Snort IDS Alerts with Windows Authentication Logs*	Provide lightweight real-time host–network correlation for academic environments	Host + Network (Unified)	Timestamp alignment + Event normalization + Snort ↔ Windows Log linkage	Snort IDS, Windows Event Viewer, PowerShell, PHP, MySQL, IIS	Simulated attack scenarios (ICMP, SYN scan, XSS, SQLi, failed logins)	No (lightweight by design)	Lightweight; real-time dashboard; Windows integration; VM-friendly; educational; simple deployment	Not enterprise scale; limited to simulated datasets; no ML

Summary

The literature reviewed in this chapter demonstrates that effective detection of modern cyber threats requires host–network integration, log correlation, and multi-layer visibility. While existing systems have made significant advancements in these areas, many remain unsuitable for lightweight deployments due to complexity, infrastructure requirements, or incomplete coverage of access-related events. These limitations reinforce the need for a practical and deployable solution such as the Access Tracing system, which focuses on unified access monitoring and traceability within a controlled and educational environment.

CHAPTER 3 – METHODOLOGY AND WORK PLAN

3.1 Introduction

This chapter presents the methodology and structured work plan adopted for the development of the Access Tracing system. Because the project integrates multiple independent components including network intrusion detection, host-level auditing, log ingestion, database storage, and dashboard visualization a clear and disciplined development methodology was required to ensure correctness, consistency, and traceability throughout the system lifecycle. The methodology defines how the system was planned, designed, implemented, tested, and documented in a sequential manner.

A well-defined methodology is particularly important in security-related projects, as improper sequencing or incomplete validation may lead to unreliable results or inaccurate interpretations of system behavior. For this reason, the Access Tracing project follows a structured development approach that emphasizes clear phase separation, early requirement definition, controlled implementation, and systematic validation. This chapter explains the selected development model, the techniques used to gather system requirements, the project schedule, and the tools and technologies employed during development.

The overall system architecture illustrating the interaction between Snort IDS, Windows Server logs, the ingestion layer, the database, and the administrative dashboard is shown in Figure 3.1.

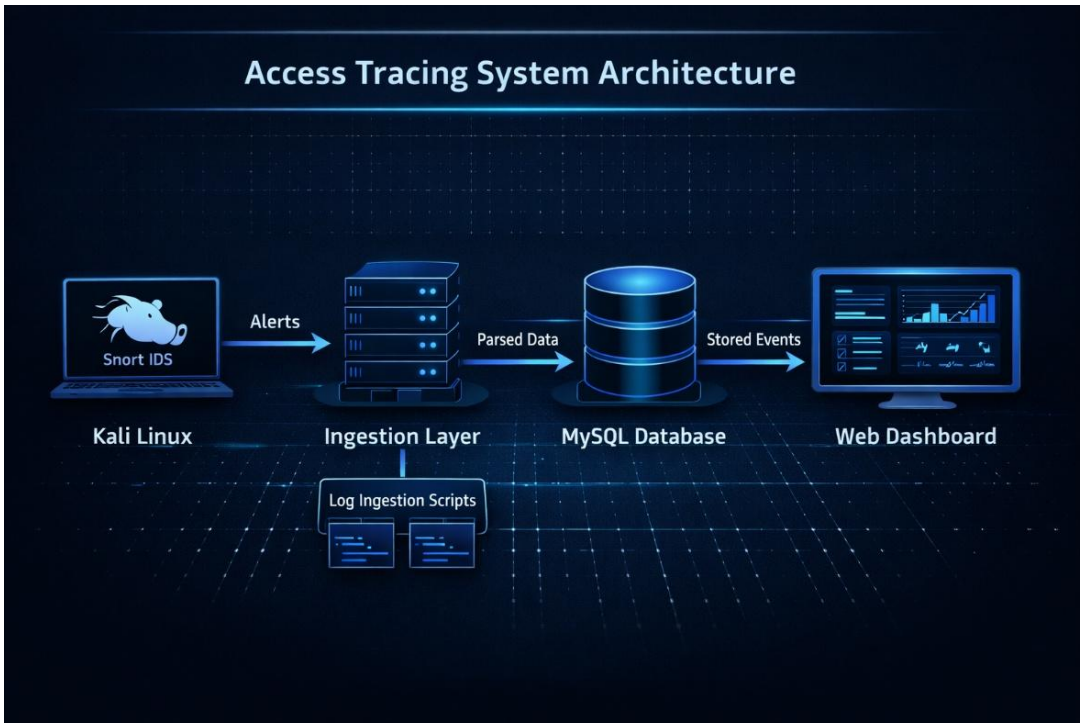

Fig 3.1: Access Tracing System Architecture

3.2 System Development Methodology (Waterfall Model)

The Access Tracing system was developed using the **Waterfall Development Model**, a linear and sequential methodology in which each phase of development is completed before moving to the next. This model was selected because the system requirements, architecture, and operational objectives were clearly defined at the beginning of the project and remained stable throughout development. The Waterfall model provides strong structure, documentation clarity, and traceability, making it suitable for systems that require reliability and disciplined implementation.

In the Waterfall approach, development progresses through distinct phases: requirement analysis, system design, implementation, testing, deployment, and documentation. Each phase produces well-defined deliverables that serve as inputs for the subsequent phase. This ensured that the Access Tracing system was built in a controlled and predictable manner, reducing development risks and preventing uncontrolled changes during later stages.

The Waterfall model also supports clear evaluation, as system behavior can be verified against predefined requirements. This was essential for validating access tracing behavior, ensuring that detection, ingestion, storage, and visualization operated exactly as specified. The sequential structure of the methodology adopted for this project is illustrated in Figure 3.2.

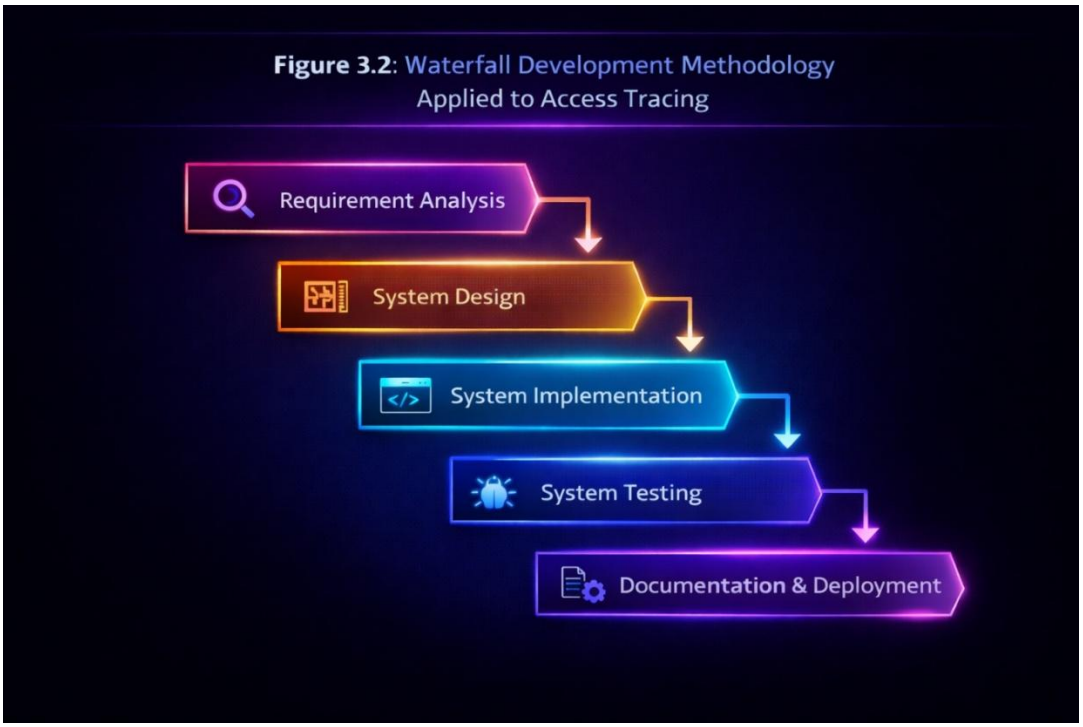

Fig 3.2: Waterfall Development Methodology Applied to Access Tracing

3.2.1 Requirement Analysis

The requirement analysis phase focused on identifying and documenting all functional and operational requirements of the Access Tracing system. During this phase, the objectives of the system were clearly defined, including the need to detect network-based attacks, capture host-level authentication activity, correlate events across layers, and present results in a centralized dashboard.

Key requirements included the ability to monitor network traffic using an intrusion detection system, record authentication and application-related events on the host, ingest logs automatically without manual intervention, normalize data into a structured format, and visualize events in near real time. Attack scenarios such as reconnaissance activity, failed authentication attempts, SQL Injection, and Cross-Site Scripting were identified as required validation cases.

This phase resulted in a complete requirement specification that guided all subsequent design and implementation decisions. No implementation work was initiated before requirements were finalized, ensuring alignment between system objectives and technical execution.

3.2.2 System Design

Following requirement analysis, the system design phase translated functional requirements into concrete technical architecture. During this phase, the overall structure of the Access Tracing system was defined, including component roles, data flow paths, and integration mechanisms. The design specified how Snort IDS would generate alerts, how Windows Server logs would be collected, how ingestion scripts would retrieve and normalize data, and how records would be stored and retrieved from the database.

The database schema was designed to support structured storage of alerts and logs, enabling efficient querying and correlation. API endpoints were designed to allow the dashboard to retrieve data securely and consistently. User interface layouts were planned to ensure clarity, readability, and effective monitoring of access events.

All design decisions were documented before implementation began, ensuring that development followed a fixed blueprint and preventing architectural inconsistencies.

3.2.3 Implementation

The implementation phase involved developing each system component according to the approved design specifications. Snort IDS was installed and configured on the Kali Linux machine to monitor network traffic and generate alerts for suspicious activity. Custom detection rules were implemented to identify reconnaissance behavior, SQL Injection attempts, and Cross-Site Scripting payloads.

On the host side, Windows Server auditing policies were enabled to record authentication attempts and relevant security events. PowerShell scripts were developed to extract logs

automatically, format them consistently, and prepare them for ingestion. A MySQL database was created to store normalized event data using predefined schemas.

The dashboard and backend services were implemented using PHP and web technologies hosted on IIS. These components enabled retrieval, filtering, and display of correlated access events. Implementation strictly followed the design documents produced in the previous phase, ensuring consistency and traceability.

3.2.4 Testing

Once implementation was completed, the system entered the testing phase. Testing focused on validating system functionality, accuracy, and reliability under controlled conditions. Predefined attack scenarios were executed to verify detection and traceability. These scenarios included network reconnaissance, authentication failures, SQL Injection attacks, and Cross-Site Scripting payload execution.

Testing verified that Snort IDS generated appropriate alerts, that Windows Server recorded corresponding authentication or application events, that ingestion scripts correctly parsed and stored logs, and that the dashboard accurately reflected recorded activity. Errors identified during testing were corrected without altering system requirements or architecture.

Evidence collected during testing including logs, database records, and dashboard screenshots was preserved for analysis and documentation.

3.2.5 Deployment and Documentation

The final phase of the Waterfall methodology involved system deployment and documentation. The Access Tracing system was stabilized within the virtual laboratory environment, ensuring that all components operated correctly together. Configuration settings were finalized, and unnecessary debugging components were removed.

Comprehensive documentation was prepared to describe system architecture, configuration steps, testing procedures, and results. Screenshots, diagrams, and logs were organized to support validation and discussion. This documentation ensures that the system can be understood, reproduced, and evaluated accurately.

3.3 Requirement Gathering Techniques

System requirements were gathered using structured and non-iterative techniques appropriate for the Waterfall methodology. The primary technique was document analysis, which involved reviewing official Snort documentation, Windows Server auditing guidelines, database design references, and prior research related to intrusion detection and access monitoring. This provided a theoretical foundation for identifying required event types and correlation needs.

Laboratory observation was used to validate documented requirements. Controlled attack scenarios were executed to observe how Snort and Windows Server generated logs. These

observations confirmed the feasibility of correlating network and host-level events and informed the design of ingestion and normalization processes.

All requirements were finalized before implementation began, ensuring consistency with the Waterfall development model. Laboratory observation was then used to validate the documented requirements through controlled attack scenarios. The overall testing and validation workflow followed during this phase is illustrated in **Figure 3.3**.

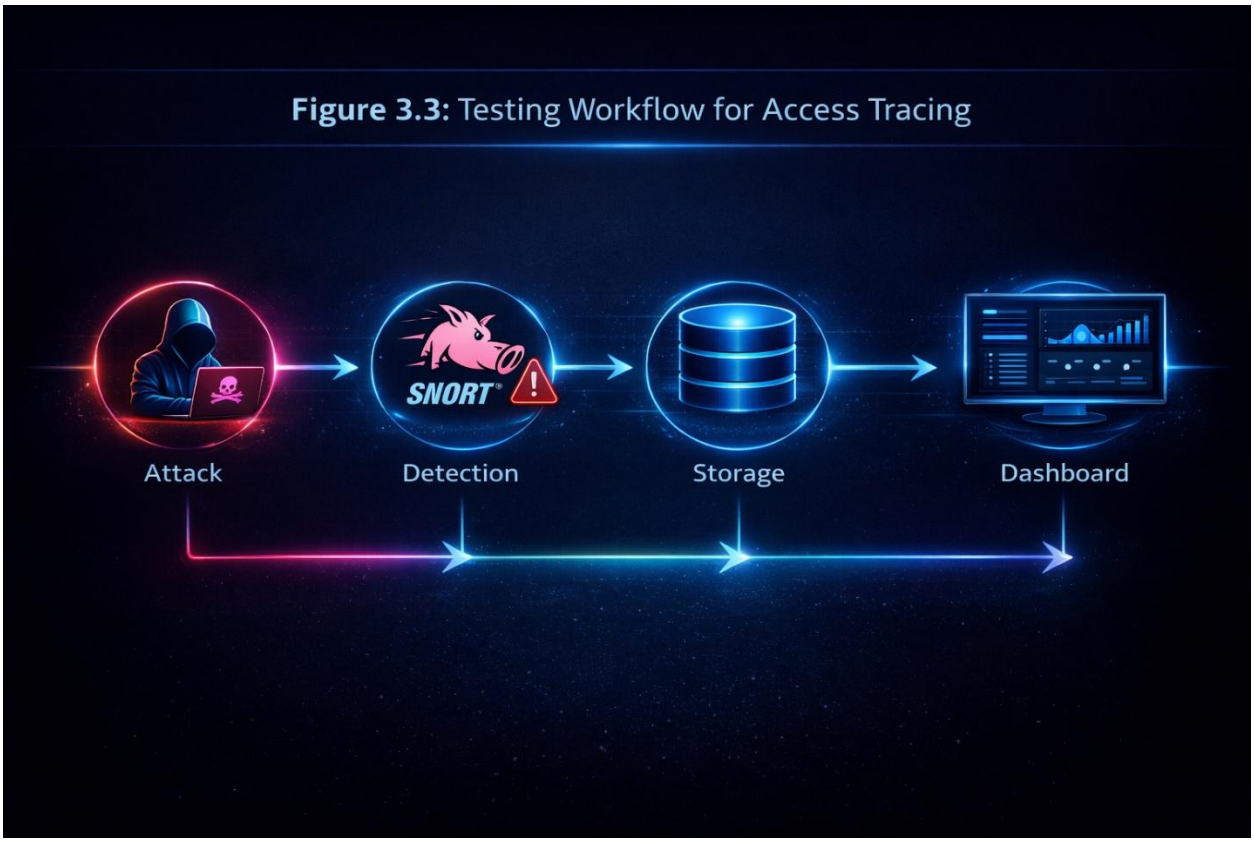

Fig 3.3: Testing Workflow for Access Tracing

3.4 Project Plan

The Access Tracing project followed a structured development schedule aligned with the Waterfall methodology. The initial phase focused on requirement analysis and documentation review. The second phase involved system design, including architecture definition and database modeling. The third phase covered implementation of Snort IDS and Windows Server auditing. The fourth phase focused on developing ingestion scripts and database integration. The fifth phase involved dashboard implementation. The sixth phase was dedicated to system testing using controlled attack scenarios. The final phase focused on documentation and result analysis.

This sequential plan ensured that each stage was completed and validated before proceeding to the next. The project timeline and phase dependencies are illustrated in Figure 3.4.

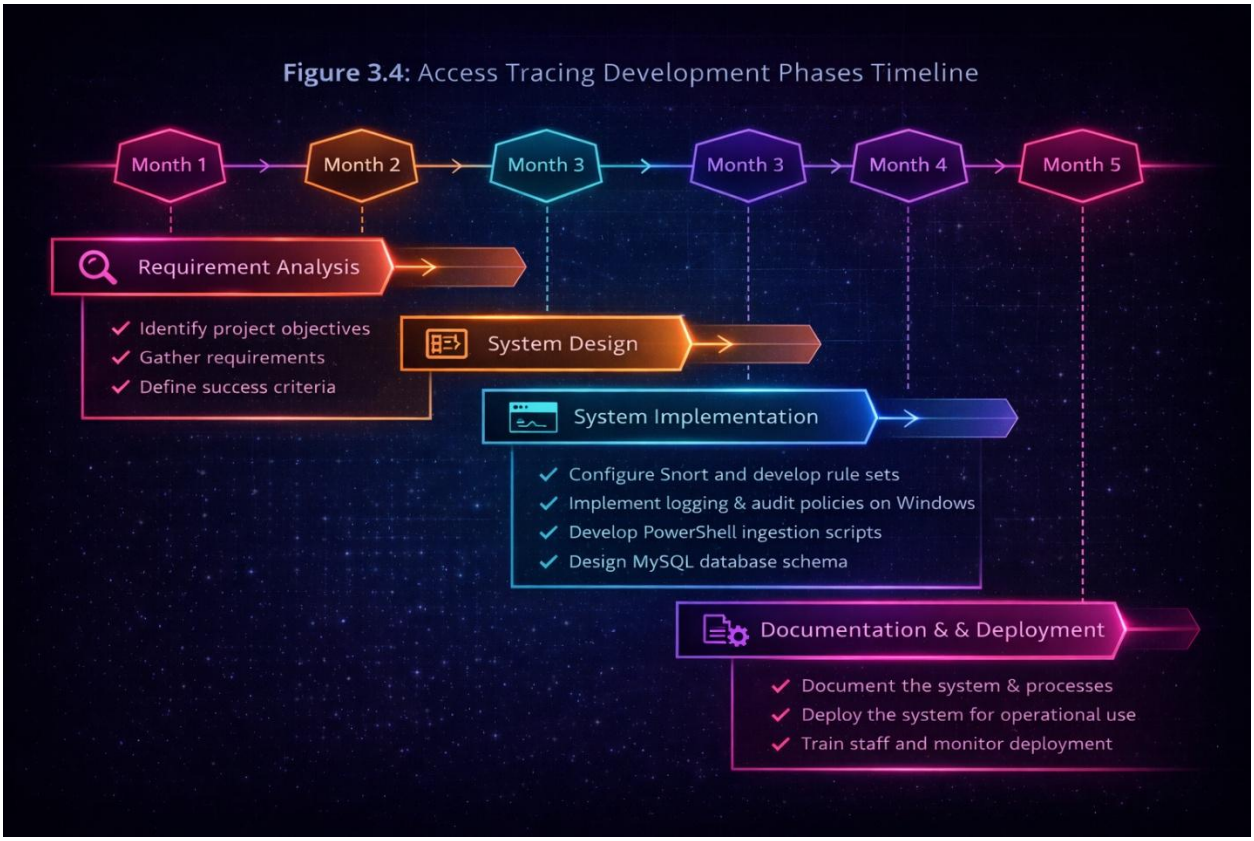

Figure 3.4: Access Tracing Development Phases Timeline

3.5 Development of Tools and Technologies

The development of the Access Tracing system utilized a combination of programming languages, frameworks, databases, and supporting technologies selected to ensure reliability, compatibility, and ease of integration. PowerShell was employed to extract Windows Security Logs using native command-line tools, enabling precise filtering, formatting, and automated retrieval of authentication events without manual intervention. PHP was used to implement the ingestion API and backend logic of the dashboard due to its seamless compatibility with Internet Information Services (IIS) and its lightweight handling of HTTP requests and MySQL database operations. JavaScript was integrated to enhance dashboard interactivity by enabling asynchronous updates, dynamic table rendering, and real-time visualization of security events.

For visualization and detection functionality, the Chart.js library was adopted to represent alert data through charts and timelines in a clear and academically appropriate manner. The Snort IDS rule engine served as the network intrusion detection layer, where custom rules were configured to identify reconnaissance activity, SQL injection attempts, and cross-site scripting

attacks, allowing flexible rule tuning for experimental validation. MySQL was selected as the central database for storing normalized intrusion alerts and host-level logs due to its reliability, structured querying capabilities, and seamless integration with PHP-based applications.

The web interface was developed using HTML to structure the dashboard layout, including tables, forms, search components, and visualization panels, while CSS was used to style the interface with a professional and readable design suitable for monitoring purposes. Internet Information Services (IIS) was used as the hosting platform to deploy the dashboard and backend services, providing stable execution and tight integration with the Windows Server environment. Visual Studio Code served as the primary development environment for writing and managing PowerShell scripts, PHP backend components, and JavaScript-based user interface logic. Windows Event Viewer and the Snort command-line interface were extensively used throughout development to monitor, test, and validate real-time log generation and system behavior during configuration and troubleshooting.

CHAPTER 4 – PROJECT DESIGN AND SPECIFICATION

4.1 Introduction

This chapter presents the detailed design and system specification of the Access Tracing project. The purpose of this chapter is to define how the system is structured, what functions it performs, and the operational constraints under which it operates. System specification plays a critical role in translating conceptual objectives into concrete technical requirements that guide implementation and validation. In the context of Access Tracing, this chapter focuses on defining the monitoring workflow starting from network traffic observation, continuing through log collection and ingestion, and ending with centralized storage and visualization.

The Access Tracing system is designed to provide end-to-end visibility into access attempts by correlating network-level intrusion detection alerts with host-level authentication and application events. This requires carefully structured architecture that ensures accurate detection, reliable data ingestion, consistent storage, and clear presentation. The specifications outlined in this chapter ensure that all components operate cohesively and that the system fulfills its intended monitoring and tracing objectives. The overall end-to-end architecture of the Access Tracing system, including network traffic detection, log ingestion, centralized storage, and dashboard visualization, is illustrated in **Figure 4.1**.

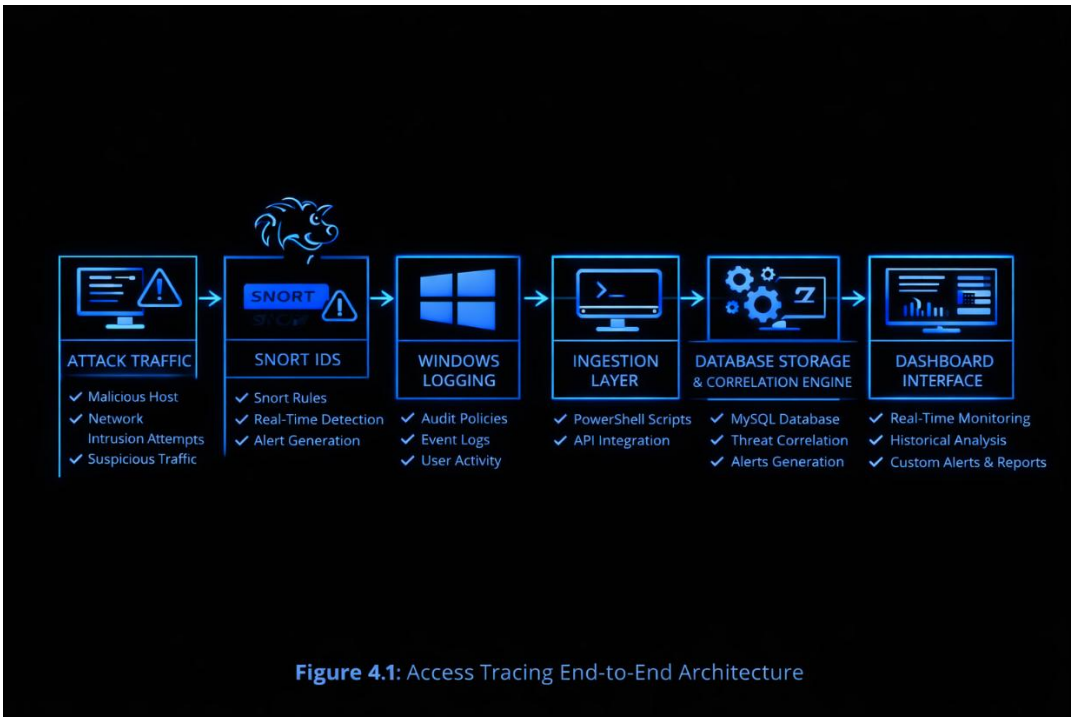

Fig 4.1: Access Tracing End-to-End Architecture

4.2 System Specification

4.2.1 Functional Requirements

The Access Tracing system is required to perform a set of functional operations that collectively enable the detection, tracing, and visualization of access-related security events. At the network layer, the system must continuously monitor inbound and outbound traffic and generate alerts when predefined attack patterns or suspicious behaviors are detected. These alerts must be generated automatically without manual intervention and must include sufficient metadata such as timestamps, source and destination addresses, and alert descriptions.

At the host layer, the system must record authentication-related events and relevant application-level activities. This includes successful and failed login attempts as well as interactions triggered by application inputs. These events must be captured reliably through the operating system's native logging mechanisms and made available for ingestion.

The system must include an automated ingestion mechanism capable of retrieving network alerts and host logs at regular intervals. This mechanism must parse raw log files, normalize their structure, and insert records into a centralized database using a predefined schema. The database must support structured storage of events and allow retrieval based on time, type, and content.

The Access Tracing system must provide an administrative dashboard that retrieves stored data through controlled interfaces and presents it in a readable and organized manner. The dashboard must display recent alerts, historical records, and aggregated views that support monitoring and review. Administrative access must be restricted through authentication mechanisms to prevent unauthorized viewing of security data.

Each functional capability of the system is designed to be testable and verifiable through practical execution, allowing validation from attack detection to dashboard visualization.

4.2.2 Non-Functional Requirements

In addition to functional capabilities, the Access Tracing system must satisfy several non-functional requirements to ensure stability, security, usability, and maintainability. From a performance perspective, the system must support near real-time availability of alerts, allowing newly generated intrusion detection events to be ingested and displayed on the dashboard with minimal delay. The system must continue to operate correctly as alert volume increases within the limits of the virtual environment.

From a security perspective, the system must restrict access to administrative interfaces through authentication mechanisms and ensure that sensitive configuration details such as database credentials are not exposed through the user interface or API responses. Data exchanged between components must follow controlled formats to prevent unauthorized data leakage.

Usability is a critical requirement, as the dashboard must present security events in a clear and intuitive layout. Administrators must be able to review alerts without complex navigation or excessive configuration. Reliability is ensured through error handling mechanisms within ingestion scripts and backend services, allowing the system to continue functioning even when logs are temporarily unavailable or incomplete.

Maintainability is achieved by separating system components into modular units, including detection rules, ingestion scripts, database schemas, and presentation layers. This modular design enables future enhancements, such as additional detection rules or extended logging support, without requiring redesign of the entire system.

4.3 Database Design

The database design of the Access Tracing system is structured to support consistent storage and retrieval of security events originating from different sources. The database schema includes tables for storing network intrusion detection alerts and host-level logs, with fields designed to capture timestamps, alert messages, source information, and event types.

Normalization techniques are applied to ensure data consistency and reduce redundancy. Each event record is uniquely identifiable, enabling efficient querying and correlation. The schema supports filtering, sorting, and aggregation operations required by the dashboard and reporting components.

4.4 User Interface Design

The user interface of the Access Tracing system is designed to support efficient monitoring and analysis of access events. The dashboard presents alerts in tabular format with clear labeling and consistent styling. Visual separation between different event types improves readability and reduces cognitive load for the administrator.

The interface includes real-time updates that reflect newly ingested alerts without requiring page reloads. Design choices emphasize clarity and professionalism, ensuring that security data is presented in a manner suitable for monitoring and documentation.

4.5 Chapter Summary

This chapter defined the design and specification of the Access Tracing system by outlining its functional and non-functional requirements, architectural structure, database design, and user interface considerations. The specifications provided in this chapter serve as a foundation for the implementation and testing phases discussed in subsequent chapters. By clearly defining system behavior and constraints, this chapter ensures that the Access Tracing system operates reliably and fulfills its objective of unified access monitoring and tracing.

CHAPTER 5 – IMPLEMENTATION

5.1 Introduction

This chapter describes the practical implementation of the Access Tracing system and explains how the designed components were deployed, configured, and integrated to achieve unified access monitoring. The implementation phase translates the architectural and specification decisions presented in the previous chapters into a working system capable of detecting, tracing, and visualizing access-related security events. The focus of this chapter is on real execution rather than theoretical design, demonstrating how network-level detection, host-level logging, automated ingestion, centralized storage, and dashboard visualization operate together as a single system.

All implementation activities were conducted within a controlled virtual laboratory environment. The system was validated using deliberate and controlled attack scenarios to ensure that each component behaved as expected and that access events could be traced from their point of origin through to final presentation on the dashboard.

5.2 Virtual Environment Setup

The Access Tracing system was implemented using a virtualized environment to ensure isolation, repeatability, and controlled testing conditions. The environment consisted of a Kali Linux virtual machine responsible for network intrusion detection and a Windows Server virtual machine hosting the monitored services, database, and dashboard. VirtualBox was used as the virtualization platform, allowing the configuration of internal networking between machines while preventing external interference.

The Kali Linux machine was configured with Snort IDS to monitor traffic directed toward the Windows Server. The Windows Server hosted a web application, database services, and dashboard interface. Network connectivity between the virtual machines was verified prior to deployment to ensure that Snort could observe traffic generated during attack simulations.

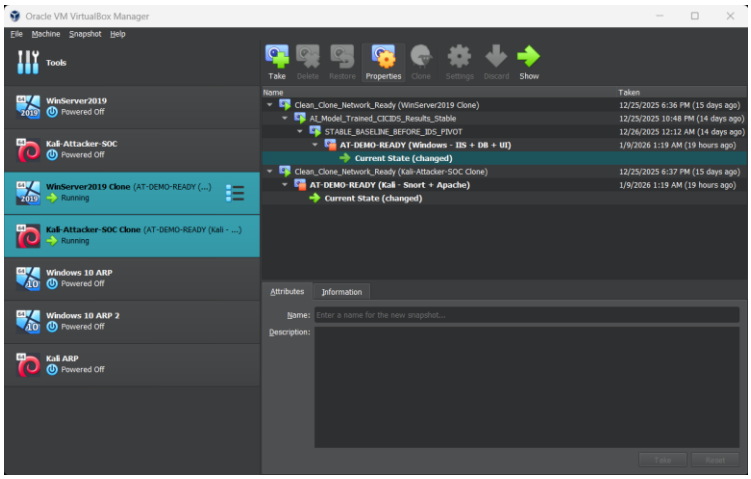

Fig 5.1.1: Virtual Laboratory Network Configuration

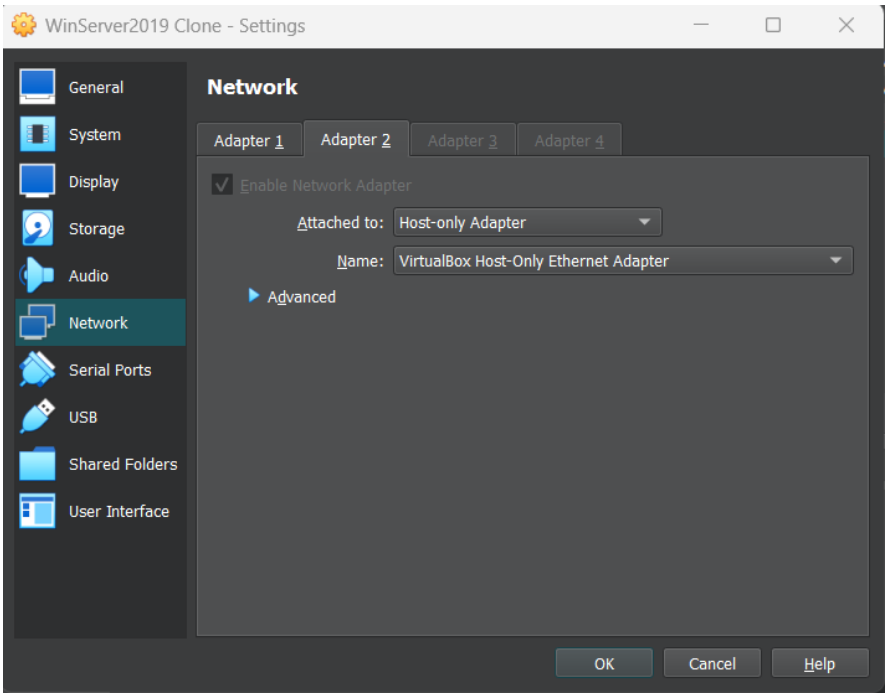

Fig 5.1.2: Virtual Laboratory Network Configuration

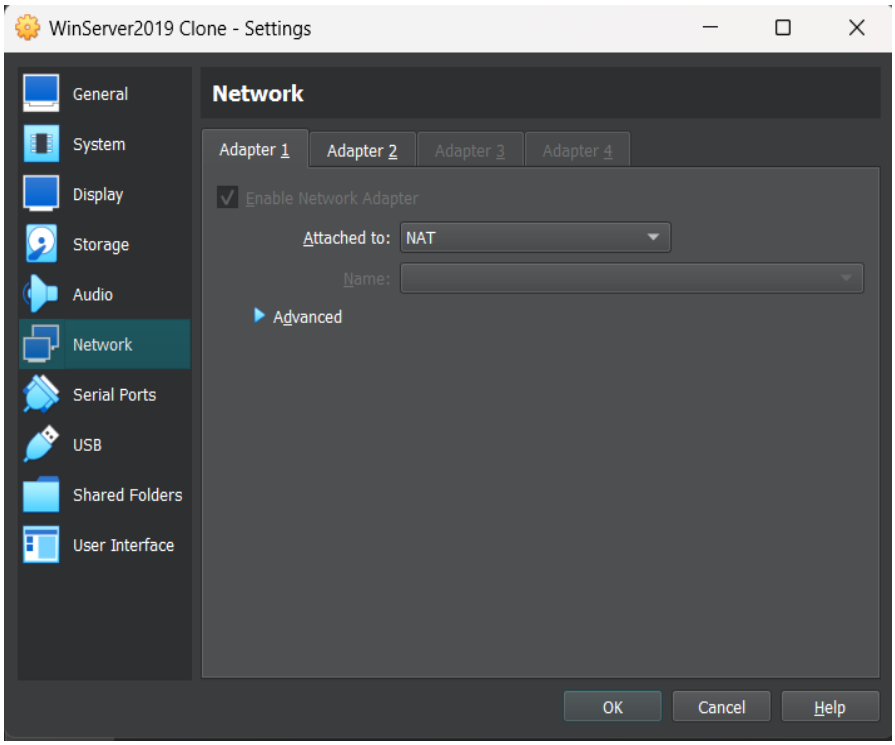

Fig 5.1.3: Virtual Laboratory Network Configuration

```
dark@kali: ~
Session  Actions  Edit  View  Help
┌──(dark㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defau
lt qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:03:ac:0f brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86333sec preferred_lft 86333sec
    inet6 fe80::a00:27ff:fe03:ac0f/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:1b:f1:16 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.20/24 brd 192.168.100.255 scope global eth1
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe1b:f116/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever

┌──(dark㉿kali)-[~]
└─$
```

Fig 5.2.1: IP Address Configuration of Kali Linux and Windows Server

```
Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.17763.3650]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::650:186e:a830:bf3e%13
   IPv4 Address. . . . . . . . . . . : 10.0.2.15
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.0.2.2

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::efb2:a063:fa5c:8951%6
   IPv4 Address. . . . . . . . . . . : 192.168.100.30
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

C:\Users\Administrator>_
```

Fig 5.2.2: IP Address Configuration of Kali Linux and Windows Server

5.3 Snort IDS Installation and Configuration

Snort was installed on the Kali Linux machine and configured to operate in network intrusion detection mode. The configuration included defining the monitored network range, specifying rule paths, and enabling alert logging. Custom rules were created to detect reconnaissance activity, SQL Injection payloads, and Cross-Site Scripting attempts based on known patterns within network traffic.

Rule testing was performed to confirm that Snort generated alerts when expected traffic patterns were observed. Alerts were written to log files in a structured format to support automated ingestion. Care was taken to ensure that alert timestamps and metadata were consistent and suitable for correlation with host-level logs.

```
┌──(dark㉿kali)-[~]
└─$ snort -V

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using libpcap version 1.10.5 (with TPACKET_V3)
           Using PCRE version: 8.39 2016-06-14
           Using ZLIB version: 1.3.1
```

Fig 5.3.1: Snort Configuration and Custom Rule Setup

```
dark@kali: ~
Session  Actions  Edit  View  Help
| Num States       : 81
| Num Match States : 9
| Memory           :   6.44Kbytes
|   Patterns       :   0.43K
|   Match Lists    :   0.91K
|   Transitions    :   3.90K
+-------------------------------------------------

MaxRss at the end of detection rules:40264

        --== Initialization Complete ==--

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using libpcap version 1.10.5 (with TPACKET_V3)
           Using PCRE version: 8.39 2016-06-14
           Using ZLIB version: 1.3.1


Total snort Fixed Memory Cost - MaxRss:40420
Snort successfully validated the configuration!
Snort exiting

┌──(dark㉿kali)-[~]
└─$
```

Fig 5.3.2: Snort Configuration and Custom Rule Setup

```
dark@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                    /etc/snort/snort.conf
# ── Minimal Snort 2.9.20 Config for Access Tracing ──

# ── Required preprocessors for HTTP content detection ──
preprocessor stream5_global: track_tcp yes, track_udp no
preprocessor stream5_tcp: policy linux, timeout 180, detect_anomalies

# HTTP normalization/inspection (lets content match URLs/headers better)
preprocessor http_inspect: global iis_unicode_map unicode.map 1252
preprocessor http_inspect_server: server default profile all ports { 80 } oversize_dir_length 500


ipvar HOME_NET 192.168.100.30
var EXTERNAL_NET any

var RULE_PATH /etc/snort/rules
var PREPROC_RULE_PATH /etc/snort/preproc_rules

include /etc/snort/reference.config
include /etc/snort/classification.config

# ── Access Tracing Output ──
output alert_fast: /var/log/snort/alert_fast.txt
#output alert_csv: AT_alerts.csv default
#output alert_full: /var/log/snort/alert

# Include ONLY your custom rules

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Fig 5.3.3: Snort IDS main configuration file

```
dark@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                    /etc/snort/rules/local.rules
# ═══════════════════════════
# Access Tracing - Custom Rules
# ═══════════════════════════
alert icmp any any → any any (msg:"AT ICMP Ping Detected"; sid:4000001; rev:1;)
alert tcp any any → any any (flags:S; msg:"AT TCP SYN Scan Detected"; sid:4000002; rev:1;)
# ── Web Visibility (Proof that Snort sees HTTP) ──
alert tcp any any → $HOME_NET 80 (msg:"AT WEB TEST: HTTP GET request seen"; flow:to_server,established; conten>
# ── SQL Injection (reliable for your sqli.php demo) ──
# Detects requests to sqli.php that contain URL-encoded single quote (%27)
# Works with payload: id=1%27%20OR%20%271%27%3D%271
alert tcp any any → $HOME_NET 80 (msg:"AT WEB SQLi Attempt (sqli.php + encoded quote %27)"; flow:to_server,est>
# Detect UNION keyword (not necessarily encoded) in the request
# Works with: 1%27%20UNION%20SELECT%201,2--%20
alert tcp any any → $HOME_NET 80 (msg:"AT WEB SQLi Attempt (UNION SELECT keyword)"; flow:to_server,established>
# ── XSS (reliable for xss.php demo) ──
# Encoded <script> tag (%3Cscript) appears in URL when you send:
# q=%3Cscript%3Ealert(1)%3C%2Fscript%3E
alert tcp any any → $HOME_NET 80 (msg:"AT WEB XSS Attempt (encoded <script>)"; flow:to_server,established; con>
# Common XSS attribute payload (often not encoded)
alert tcp any any → $HOME_NET 80 (msg:"AT WEB XSS Attempt (onerror=)"; flow:to_server,established; content:"on>
# ── DEBUG (optional) ──
# This rule is only for proving raw GET bytes exist (keep or remove after demo)
alert tcp any any → 192.168.100.30 80 (msg:"AT DEBUG RAW HTTP GET /test.php"; flow:to_server,established; cont>

                                [ Read 23 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Fig 5.3.4: Snort Rules

```
dark@kali: ~
Session  Actions  Edit  View  Help

┌──(dark㉿kali)-[~]
└─$ sudo tail -n 20 /var/log/snort/alert
01/08-16:23:54.435426  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.10
0.20:40466 -> 192.168.100.30:80
01/08-16:24:12.716370  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:54178
 -> 192.168.100.30:80
01/08-16:24:12.717515  [**] [1:4000101:3] AT WEB SQLi Attempt (sqli.php + encoded quote %27) [**] [Priority: 0]
{TCP} 192.168.100.20:54178 -> 192.168.100.30:80
01/08-16:24:12.717515  [**] [1:4000102:3] AT WEB SQLi Attempt (UNION SELECT keyword) [**] [Priority: 0] {TCP} 19
2.168.100.20:54178 -> 192.168.100.30:80
01/08-16:24:12.717515  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.10
0.20:54178 -> 192.168.100.30:80
01/09-13:33:08.311139  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {IPV6-ICMP} fe80::a00:27ff:fe
03:ac0f -> ff02::2
01/09-13:36:49.358742  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:54096
 -> 192.168.100.30:445
01/09-13:36:49.358944  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:54096
 -> 192.168.100.30:80
01/09-13:36:49.359066  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:54096
 -> 192.168.100.30:22
01/09-13:45:31.651428  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.20 -> 192.
168.100.30
01/09-13:45:31.652913  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.30 -> 192.
168.100.20
01/09-13:45:32.662881  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.20 -> 192.
168.100.30
01/09-13:45:32.664091  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.30 -> 192.
168.100.20
01/09-13:46:21.640048  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:58680
 -> 192.168.100.30:80
01/09-13:46:21.641018  [**] [1:4000101:3] AT WEB SQLi Attempt (sqli.php + encoded quote %27) [**] [Priority: 0]
{TCP} 192.168.100.20:58680 -> 192.168.100.30:80
01/09-13:46:21.641018  [**] [1:4000102:3] AT WEB SQLi Attempt (UNION SELECT keyword) [**] [Priority: 0] {TCP} 19
2.168.100.20:58680 -> 192.168.100.30:80
01/09-13:46:21.641018  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.10
0.20:58680 -> 192.168.100.30:80
01/09-13:46:32.185140  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:46294
 -> 192.168.100.30:80
01/09-13:46:32.186272  [**] [1:4000103:3] AT WEB XSS Attempt (encoded <script>) [**] [Priority: 0] {TCP} 192.168
.100.20:46294 -> 192.168.100.30:80
01/09-13:46:32.186272  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.10
0.20:46294 -> 192.168.100.30:80
```

Fig 5.3.5: Snort Alert Log Monitoring

```
01/08-16:24:12.717515  [**] [1:4000101:3] AT WEB SQLi Attempt (sqli.php + encoded quote %27) [**]
[Priority: 0] {TCP} 192.168.100.20:54178 -> 192.168.100.30:80
01/08-16:24:12.717515  [**] [1:4000102:3] AT WEB SQLi Attempt (UNION SELECT keyword) [**] [Priorit
y: 0] {TCP} 192.168.100.20:54178 -> 192.168.100.30:80
```

Fig 5.3.6: Snort Alert Triggered by SQL Injection

```
01/09-13:46:32.186272  [**] [1:4000103:3] AT WEB XSS Attempt (encoded <script>) [**] [Priority: 0] {TCP} 192.168
.100.20:46294 -> 192.168.100.30:80
```

Figure 5.3.7: Snort Alert Triggered by XSS Attack

5.4 Windows Server Auditing and Web Application Deployment

On the Windows Server, auditing policies were enabled to ensure that authentication events and relevant system activities were recorded. These policies allowed the system to capture failed login attempts, successful authentications, and application-related interactions that resulted from user input.

A deliberately vulnerable web application was deployed using Internet Information Services (IIS). The application accepted user input through web forms and interacted with a backend database. This setup enabled controlled execution of SQL Injection and Cross-Site Scripting attacks for monitoring and tracing purposes. The application was intentionally left without input sanitization to allow exploitation in a controlled manner.

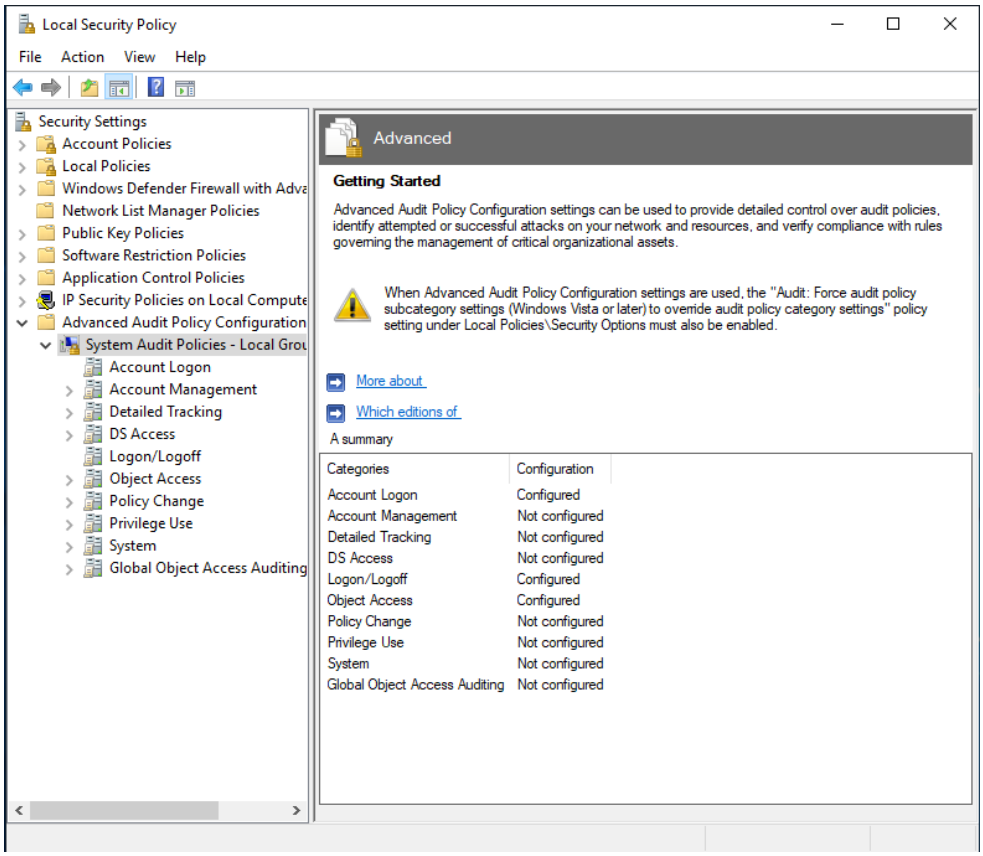

Fig 5.4.1: Windows Server Advanced Audit Policy Configuration

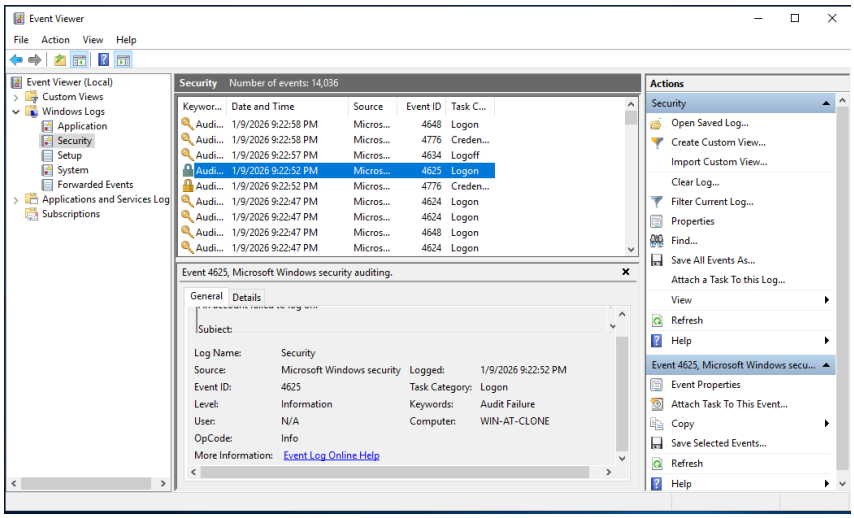

Fig 5.4.2: Windows Security Authentication Events

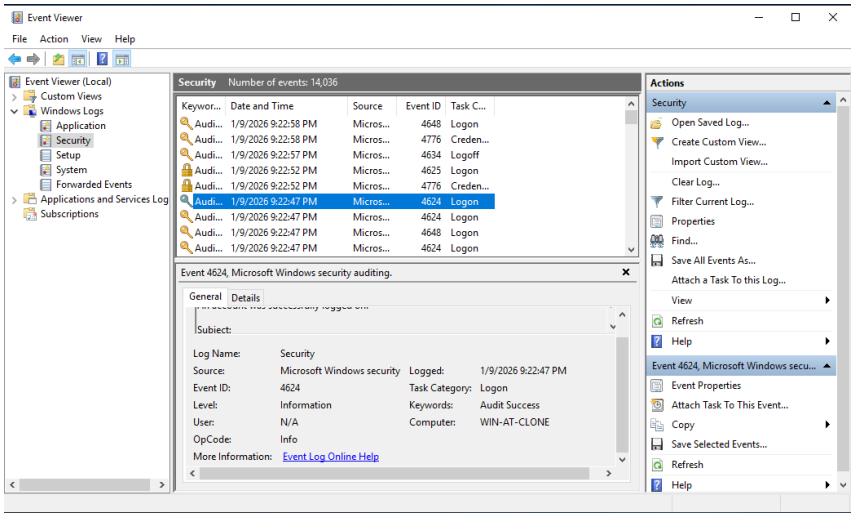

Fig 5.4.3: Windows Security Authentication Events

5.5 SQL Injection Attack Execution

SQL Injection attacks were executed against the deployed web application by submitting crafted input designed to manipulate backend database queries. These inputs allowed unauthorized behavior such as bypassing authentication or altering query logic. Upon submission, the application processed the payload and generated observable effects, confirming successful exploitation.

From a monitoring perspective, the SQL Injection attack generated network traffic detected by Snort and application-level activity recorded on the Windows Server. Snort rules matched SQL-related patterns within the payload, producing intrusion alerts. Simultaneously, the web application and database interaction generated host-level evidence suitable for correlation.

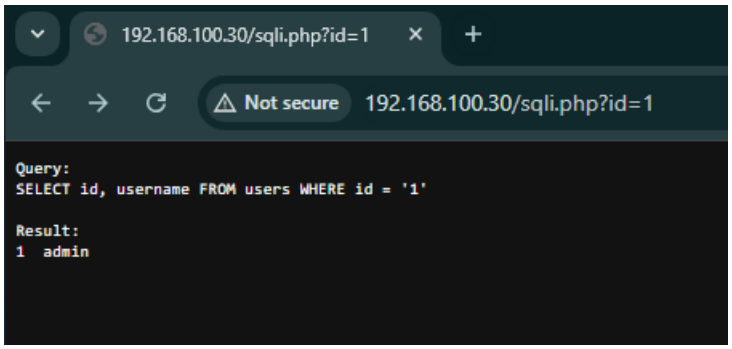

Fig 5.5.1: Normal Web Application Response

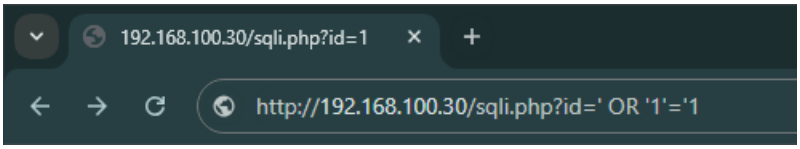

Fig 5.5.2: SQL Injection Payload Submission

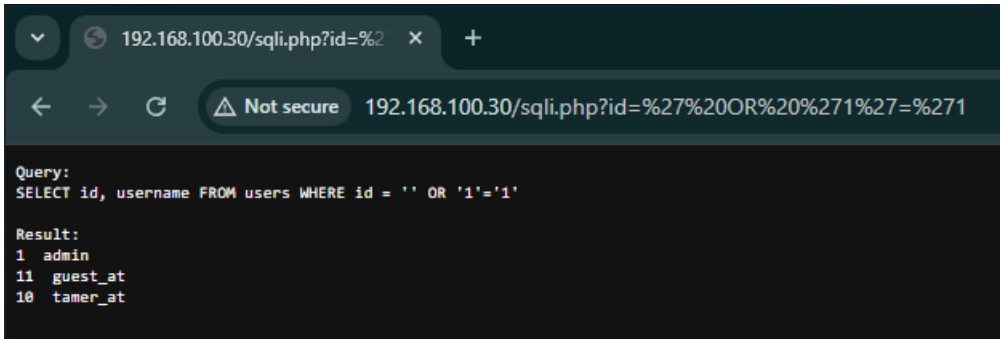

Fig 5.5.3: Application Response After SQL Injection

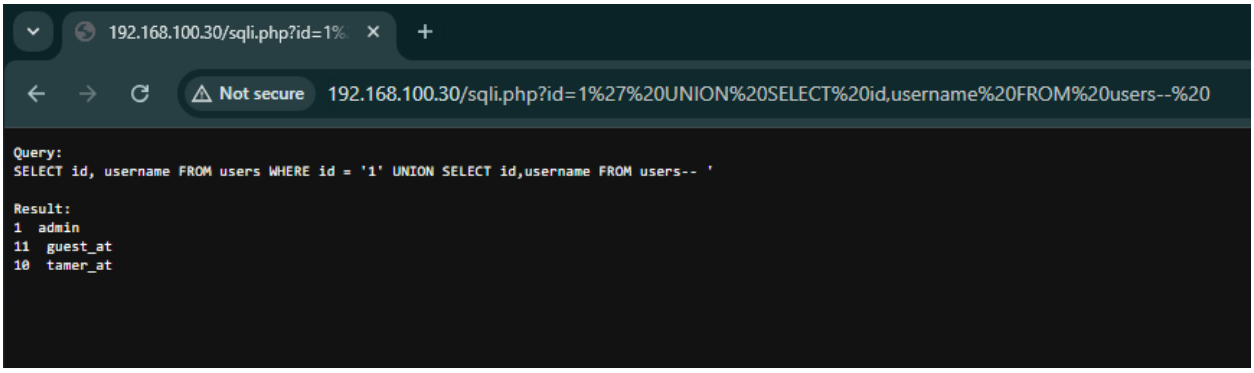

Fig 5.5.4: SQL Injection UNION SELECT

5.6 Cross-Site Scripting (XSS) Attack Execution

Cross-Site Scripting attacks were performed by injecting script-based payloads into the web application's input fields. When the application rendered the input, the script executed in the client browser, confirming the presence of a reflected XSS vulnerability. This execution demonstrated how malicious input could affect users interacting with the application.

The injected payload generated network traffic observable by Snort and application-level logs on the Windows Server. Snort rules designed to detect script patterns within HTTP traffic generated alerts corresponding to the XSS attempt. These alerts provided network-level evidence that complemented host-level records.

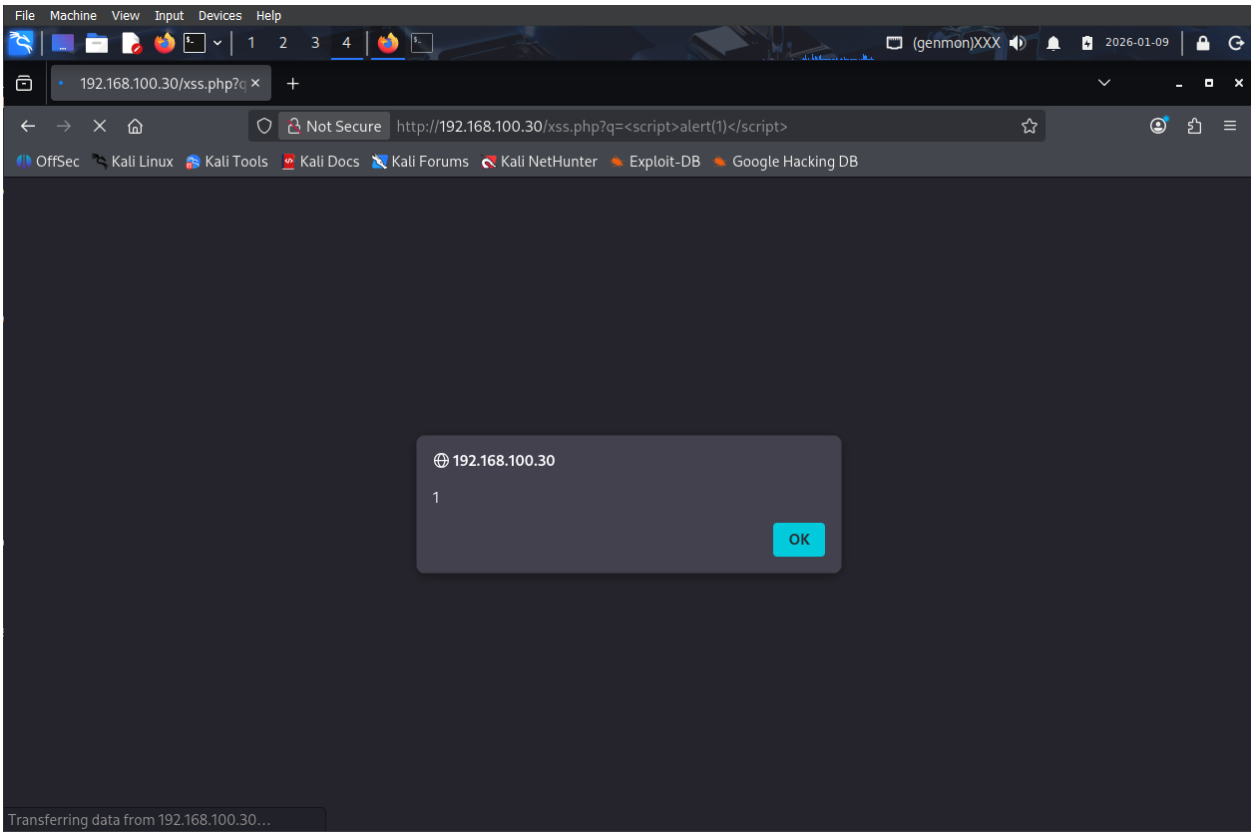

Fig 5.6.1: Reflected XSS Payload Injection via URL Parameter

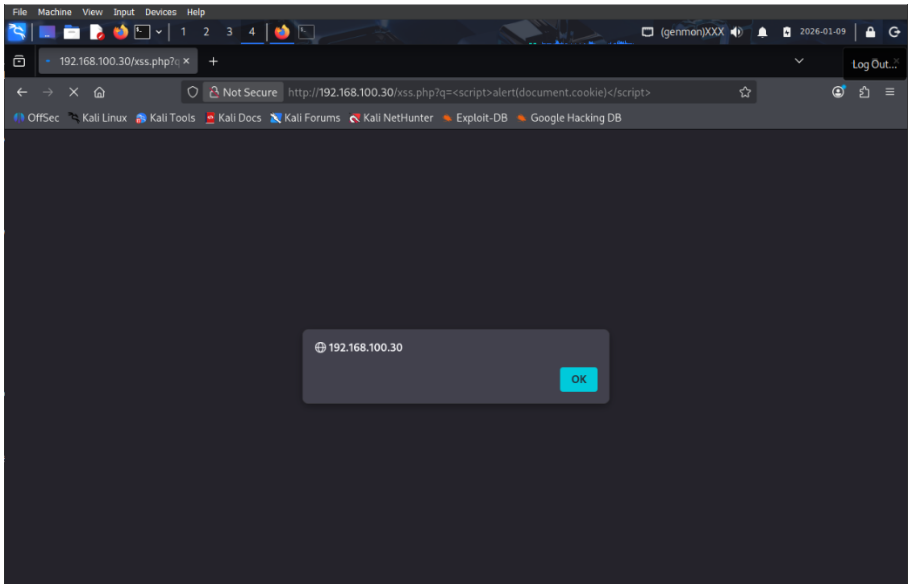

Fig 5.6.2: XSS Exploitation – Client-Side Data Access Attempt

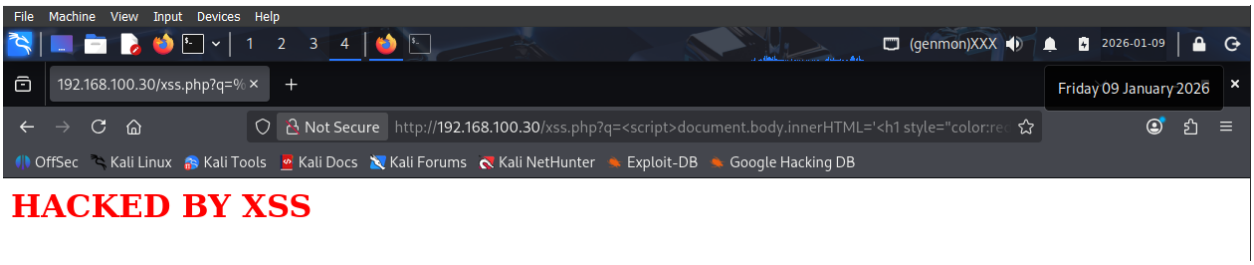

Fig 5.6.3: XSS-Based Page Manipulation (DOM Modification)

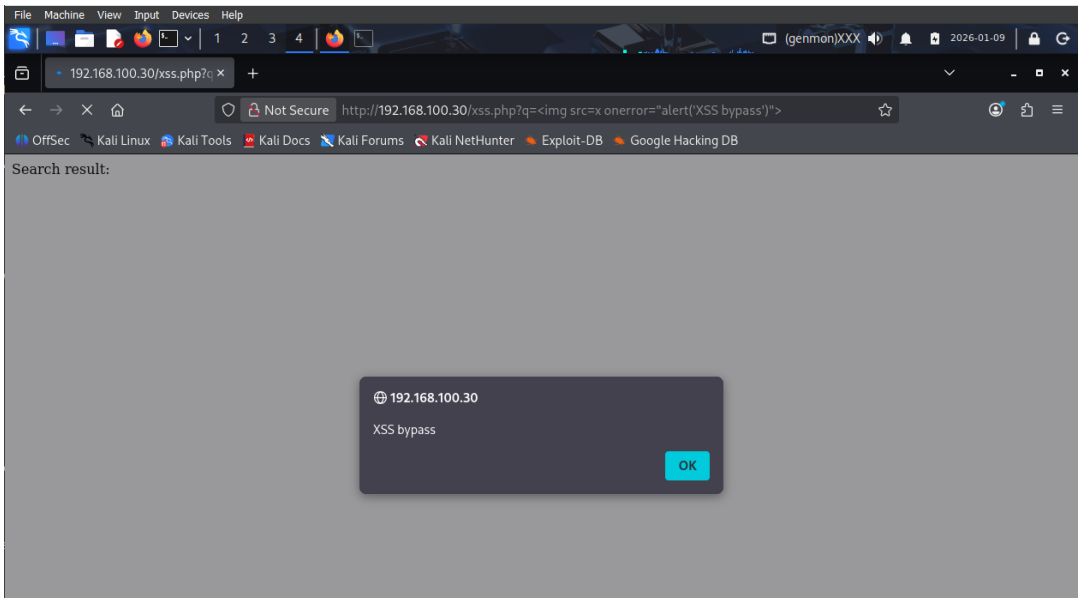

Fig 5.6.4: XSS Filter Bypass Using HTML Event Handler

5.7 Log Ingestion and Data Normalization

Automated ingestion scripts were implemented to retrieve Snort alert files and Windows Server logs at regular intervals. These scripts parsed raw logs, extracted relevant fields, normalized timestamps, and prepared records for database insertion. PowerShell was used to extract host-level logs, while PHP handled network alert ingestion and API exposure.

Normalization ensured that data originating from different sources followed a consistent structure, enabling reliable storage and querying. Error handling mechanisms were included to prevent ingestion failures from interrupting system operation.

```
Administrator: Windows PowerShell
[MYSQL ERROR] mysql: [Warning] Using a password on the command line interface can be insecure. ERROR 1045 (28000): Acces
s denied for user '$dbUser'@'localhost' (using password: YES)
PS C:\Users\Administrator> dir C:\MyProject\*.ps1 | Select Name
>>

Name
----
ingest_event_logs.ps1
ingest_raw.ps1
ingest_snort_alerts.ps1
ingest_snort_demo.ps1
ingest_snort_live.ps1


PS C:\Users\Administrator> & "C:\Program Files\MySQL\MySQL Server 9.4\bin\mysql.exe" "--user=app_user" "--password=123"
"--database=demo_app" -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+---+
| 1 |
+---+
| 1 |
+---+
PS C:\Users\Administrator> PowerShell.exe -ExecutionPolicy Bypass -File "C:\MyProject\ingest_snort_demo.ps1"
PS C:\Users\Administrator> PowerShell.exe -ExecutionPolicy Bypass -File "C:\MyProject\ingest_snort_demo.ps1"
PS C:\Users\Administrator> notepad C:\MyProject\ingest_snort_demo.ps1
PS C:\Users\Administrator> notepad C:\MyProject\ingest_snort_demo.ps1
PS C:\Users\Administrator> PowerShell.exe -ExecutionPolicy Bypass -File "C:\MyProject\ingest_snort_demo.ps1"
[OK] Lines fetched: 22
[OK] Lines matched (full parse): 22
[OK] Lines fallback-inserted: 0
[OK] Rows inserted (real): 22
[OK] MySQL errors: 0
PS C:\Users\Administrator>
```

Fig 5.7.1: PowerShell ingestion

```
Select Administrator: Windows PowerShell
PS C:\Users\Administrator> Invoke-WebRequest -Uri "http://192.168.100.20/AT_fast_alerts.log" -UseBasicParsing -OutFile "
C:\MyProject\at_fast.log"
PS C:\Users\Administrator> (Get-Item "C:\MyProject\at_fast.log").Length
3045
PS C:\Users\Administrator> Get-Content "C:\MyProject\at_fast.log" -TotalCount 20
01/08-16:23:11.552269  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.20 -> 192.168.100.
30
01/08-16:23:11.553093  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.30 -> 192.168.100.
20
01/08-16:23:12.569199  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.20 -> 192.168.100.
30
01/08-16:23:12.569845  [**] [1:4000001:1] AT ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.100.30 -> 192.168.100.
20
01/08-16:23:21.101521  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:49730 -> 192.
168.100.30:80
01/08-16:23:21.101608  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:49730 -> 192.
168.100.30:445
01/08-16:23:21.101634  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:49730 -> 192.
168.100.30:22
01/08-16:23:28.131052  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:45112 -> 192.
168.100.30:80
01/08-16:23:28.132184  [**] [1:4000998:2] AT DEBUG RAW HTTP GET /test.php [**] [Priority: 0] {TCP} 192.168.100.20:45112
-> 192.168.100.30:80
01/08-16:23:28.132184  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.100.20:451
12 -> 192.168.100.30:80
01/08-16:23:36.680362  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:43698 -> 192.
168.100.30:80
01/08-16:23:36.681306  [**] [1:4000103:3] AT WEB XSS Attempt (encoded <script>) [**] [Priority: 0] {TCP} 192.168.100.20:
43698 -> 192.168.100.30:80
01/08-16:23:36.681306  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.100.20:436
98 -> 192.168.100.30:80
01/08-16:23:44.556219  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:55482 -> 192.
168.100.30:80
01/08-16:23:44.557316  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.100.20:554
82 -> 192.168.100.30:80
01/08-16:23:54.434582  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:40466 -> 192.
168.100.30:80
01/08-16:23:54.435426  [**] [1:4000101:3] AT WEB SQLi Attempt (sqli.php + encoded quote %27) [**] [Priority: 0] {TCP} 19
2.168.100.20:40466 -> 192.168.100.30:80
01/08-16:23:54.435426  [**] [1:4000200:2] AT WEB TEST: HTTP GET request seen [**] [Priority: 0] {TCP} 192.168.100.20:404
66 -> 192.168.100.30:80
01/08-16:24:12.716370  [**] [1:4000002:1] AT TCP SYN Scan Detected [**] [Priority: 0] {TCP} 192.168.100.20:54178 -> 192.
168.100.30:80
01/08-16:24:12.717515  [**] [1:4000101:3] AT WEB SQLi Attempt (sqli.php + encoded quote %27) [**] [Priority: 0] {TCP} 19
2.168.100.20:54178 -> 192.168.100.30:80
```

Fig 5.7.2: Retrieved Snort Alert Log Containing Multiple Detected Attack Events

5.8 Database Storage Implementation

A MySQL database was used to store normalized access tracing data. Tables were created to store Snort alerts and host-level events, with fields designed to capture timestamps, alert descriptions, and source information. Successful insertion of records confirmed the integrity of the ingestion process.

Database queries were tested to ensure that stored events could be retrieved, filtered, and counted accurately. The database served as the central repository linking network and host-level evidence.

After log ingestion, the parsed Snort alerts are inserted into a centralized MySQL database. The database schema stores essential attributes such as event time, alert message, protocol, source IP, and destination IP. To verify successful storage, SQL queries were executed to count and retrieve recent records from the snort_alerts table. The results confirm that multiple attack events, including ICMP, TCP SYN scans, SQL injection, and XSS attempts, were successfully stored. The stored records are shown in Figure 5.8.

```
Administrator: C:\Windows\system32\cmd.exe - mysql  -u app_user -p
mysql> USE demo_app;
Database changed
mysql>
mysql> SELECT COUNT(*) FROM snort_alerts;
+----------+
| COUNT(*) |
+----------+
|       22 |
+----------+
1 row in set (0.005 sec)

mysql>
mysql> SELECT id, event_time, alert_message, protocol, source_ip, destination_ip
    -> FROM snort_alerts
    -> ORDER BY id DESC
    -> LIMIT 30;
+----+---------------------+-------------------------------------------------+----------+---------------+---------------+
| id | event_time          | alert_message                                   | protocol | source_ip     | destination_ip |
+----+---------------------+-------------------------------------------------+----------+---------------+---------------+
| 22 | 01/08-16:24:12.717515 | AT WEB TEST: HTTP GET request seen              | TCP      | 192.168.100.20 | 192.168.100.30 |
| 21 | 01/08-16:24:12.717515 | AT WEB SQLi Attempt (UNION SELECT keyword)      | TCP      | 192.168.100.20 | 192.168.100.30 |
| 20 | 01/08-16:24:12.717515 | AT WEB SQLi Attempt (sqli.php + encoded quote %27) | TCP      | 192.168.100.20 | 192.168.100.30 |
| 19 | 01/08-16:24:12.716370 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
| 18 | 01/08-16:23:54.435426 | AT WEB TEST: HTTP GET request seen              | TCP      | 192.168.100.20 | 192.168.100.30 |
| 17 | 01/08-16:23:54.435426 | AT WEB SQLi Attempt (sqli.php + encoded quote %27) | TCP      | 192.168.100.20 | 192.168.100.30 |
| 16 | 01/08-16:23:54.434582 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
| 15 | 01/08-16:23:44.557316 | AT WEB TEST: HTTP GET request seen              | TCP      | 192.168.100.20 | 192.168.100.30 |
| 14 | 01/08-16:23:44.556219 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
| 13 | 01/08-16:23:36.681306 | AT WEB TEST: HTTP GET request seen              | TCP      | 192.168.100.20 | 192.168.100.30 |
| 12 | 01/08-16:23:36.681306 | AT WEB XSS Attempt (encoded <script>)           | TCP      | 192.168.100.20 | 192.168.100.30 |
| 11 | 01/08-16:23:36.680362 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
| 10 | 01/08-16:23:28.132184 | AT WEB TEST: HTTP GET request seen              | TCP      | 192.168.100.20 | 192.168.100.30 |
|  9 | 01/08-16:23:28.132184 | AT DEBUG RAW HTTP GET /test.php                 | TCP      | 192.168.100.20 | 192.168.100.30 |
|  8 | 01/08-16:23:28.131052 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
|  7 | 01/08-16:23:21.101634 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
|  6 | 01/08-16:23:21.101608 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
|  5 | 01/08-16:23:21.101521 | AT TCP SYN Scan Detected                        | TCP      | 192.168.100.20 | 192.168.100.30 |
|  4 | 01/08-16:23:12.569845 | AT ICMP Ping Detected                           | ICMP     | 192.168.100.30 | 192.168.100.20 |
|  3 | 01/08-16:23:12.569199 | AT ICMP Ping Detected                           | ICMP     | 192.168.100.20 | 192.168.100.30 |
|  2 | 01/08-16:23:11.553093 | AT ICMP Ping Detected                           | ICMP     | 192.168.100.30 | 192.168.100.20 |
|  1 | 01/08-16:23:11.552269 | AT ICMP Ping Detected                           | ICMP     | 192.168.100.20 | 192.168.100.30 |
+----+---------------------+-------------------------------------------------+----------+---------------+---------------+
```

Fig 5.8: Stored Snort Alerts in MySQL Database After Log Ingestion

5.9 Dashboard Implementation and Visualization

The administrative dashboard was implemented using PHP, HTML, CSS, and JavaScript, and hosted on IIS. The dashboard retrieved data through controlled API endpoints and displayed access events in tabular and visual formats. Real-time updates allowed newly ingested alerts to appear without manual refresh. The dashboard provided administrators with a centralized view of access attempts, enabling inspection of attack type, timing, and frequency. This visualization

confirmed the effectiveness of the Access Tracing system in correlating network and host-level events. To provide a high-level overview of system activity, the dashboard includes visual summaries that aggregate detected events. As shown in figure 5.9.

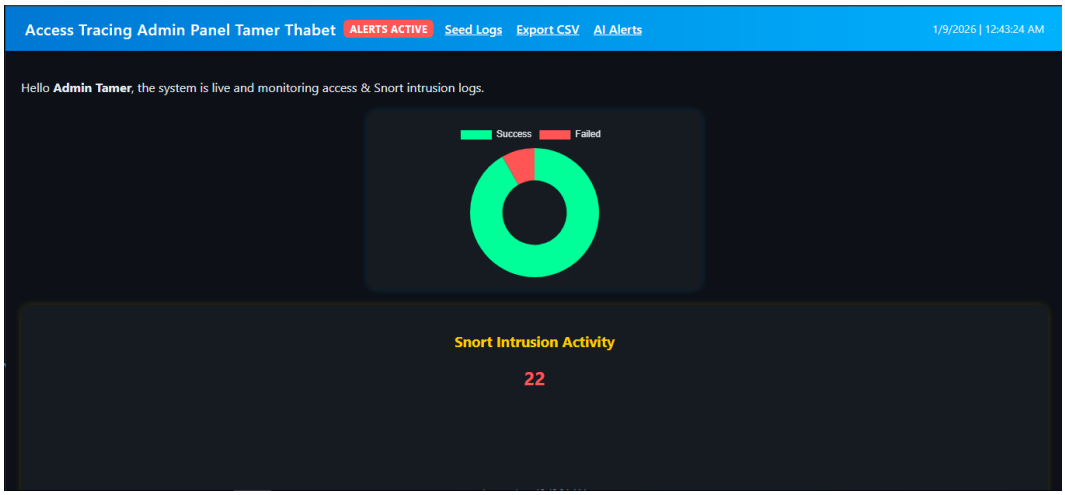

Fig 5.9.1: Dashboard Summary of Intrusion Activity

In addition to network intrusion alerts, the dashboard also displays host-level authentication events collected from the operating system. This enables correlation between intrusion attempts and user access behavior. Figure 5. illustrates successful and failed authentication events recorded by the system.

ID	USER	STATUS	IP	SOURCE	EVENT ID	HOST	SEVERITY	THREAT INTEL	TIME
12	admin	Success	192.168.100.30	web			info	-	2026-01-09 00:14:01
11	admin	Success	192.168.100.30	web			info	-	2025-12-25 19:51:37
10	admin	Success	192.168.100.10	web			info	-	2025-12-20 20:01:39
9	admin	Success	192.168.100.10	web			info	-	2025-12-13 21:31:02
8	admin	Failed	192.168.100.10	web			high	-	2025-12-13 21:30:56
7	admin	Success	192.168.100.10	web			info	-	2025-12-13 21:30:13
6	admin	Success	192.168.100.10	web			info	-	2025-11-28 20:02:58
5	admin	Success	192.168.100.10	web			info	-	2025-11-28 19:56:39
4	admin	Success	192.168.100.10	web			info	-	2025-11-25 15:23:20
3	admin	Success	192.168.100.10	web			info	-	2025-11-25 15:09:01
2	admin	Success	192.168.100.10	web			info	-	2025-11-25 14:52:06
1	admin	Success	192.168.100.10	web			info	-	2025-11-18 23:16:34

Fig 5.9.2: Dashboard View of User Authentication Events

After log ingestion and database storage, the Access Tracing system presents correlated security events through an administrative dashboard. The dashboard retrieves normalized records from the database and displays them in real time, allowing administrators to review intrusion activity and application-level attacks. Figure 5.19 shows the dashboard presenting multiple correlated Snort alerts, including SQL injection, XSS attempts, and network scans.

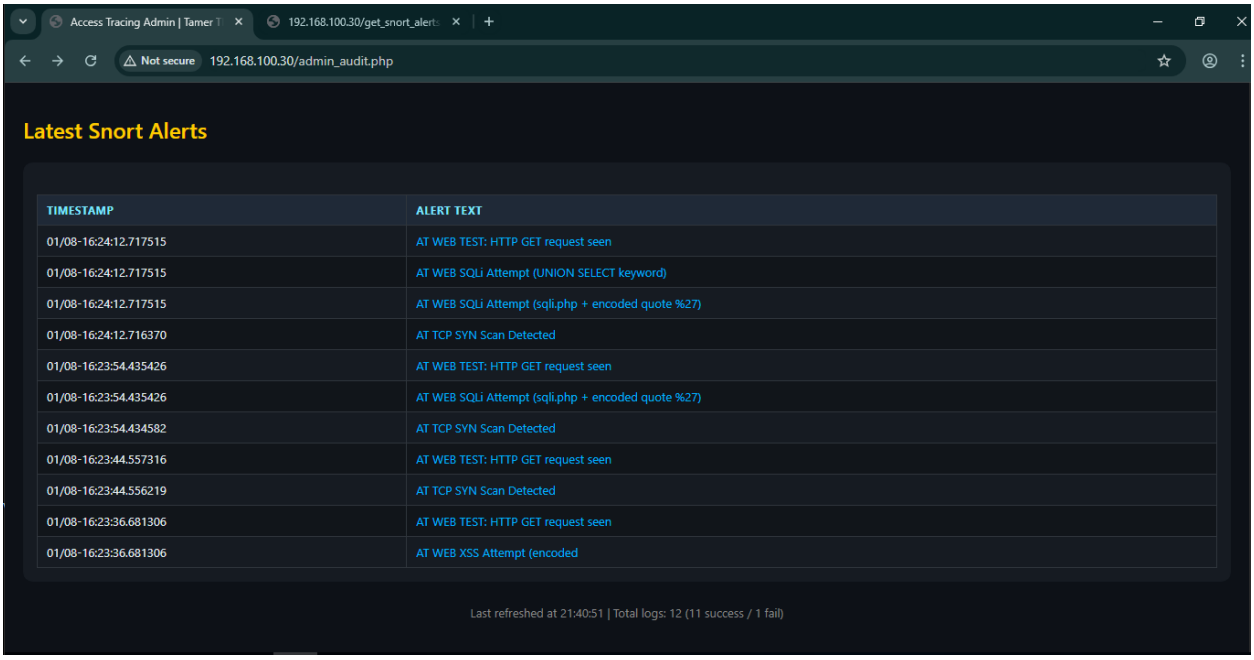

Fig 5.9.3: Admin Dashboard Displaying Correlated Snort Alerts

5.10 Chapter Summary

This chapter detailed the practical implementation of the Access Tracing system, describing how each component was deployed and validated within a controlled environment. Through deliberate execution of SQL Injection and Cross-Site Scripting attacks, the system demonstrated its ability to detect, ingest, store, and visualize access-related security events. The successful integration of network intrusion detection and host-level logging confirms the system's capability to trace access attempts end-to-end.

CHAPTER 6 – SYSTEM IMPLEMENTATION

6.1 Introduction

This chapter describes the system implementation phase of the Access Tracing project, during which the approved system design was transformed into a fully functioning monitoring system. The implementation phase focuses on translating the design models and specifications into executable components, integrating system modules, and validating correct functionality through practical execution. This chapter explains how the core components of the system were implemented, how they interact with each other, and how access tracing functionality was achieved in practice.

The implementation phase represents the transition from conceptual design to operational reality. All system components, including intrusion detection, log ingestion, database integration, and user interface elements, were implemented according to the specifications defined in earlier chapters. Initial functional testing was conducted during implementation to verify correctness, stability, and integration between system components. The testing process followed to evaluate the Access Tracing system, starting from attack execution through detection, ingestion, storage, and dashboard visualization, is illustrated in **Figure 6.1**.

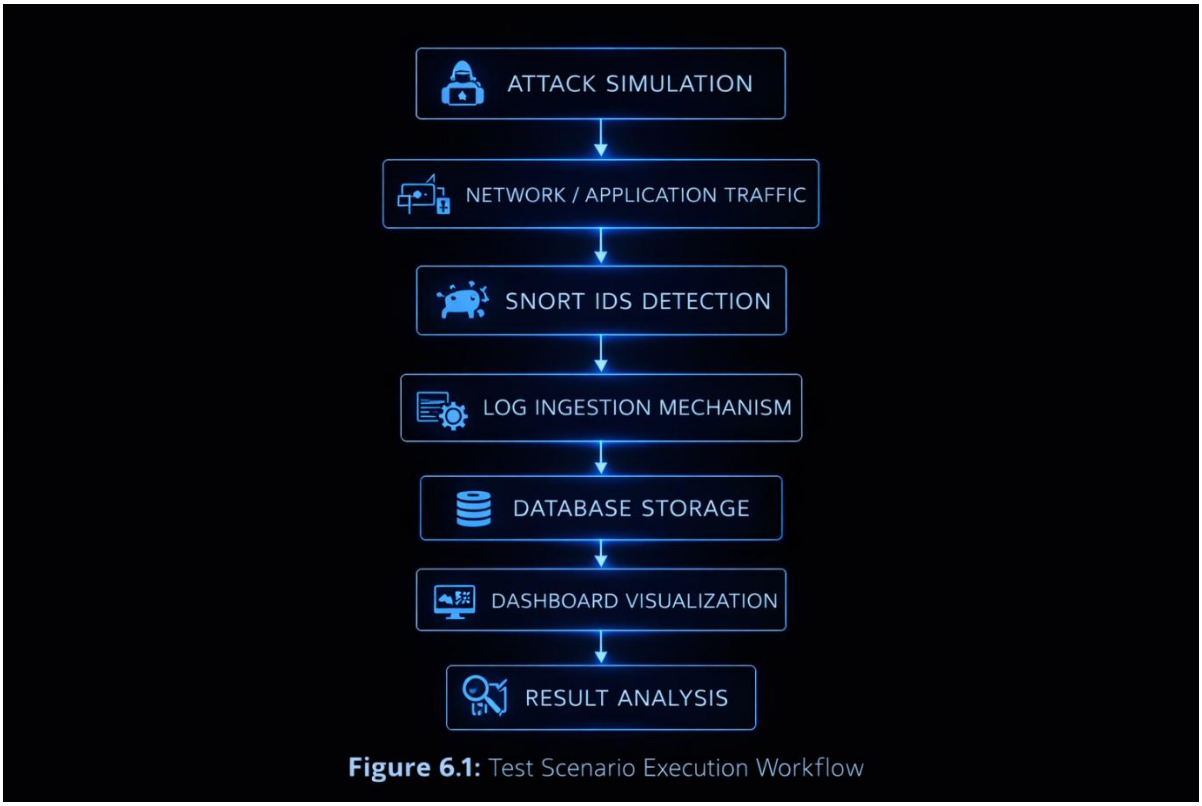

Fig 6.1: Test Scenario Execution Workflow

6.2 Implementation Overview

The implementation of the Access Tracing system involved a structured process in which the system design was translated into source code and operational components. The process began with the configuration of the network intrusion detection engine and the preparation of the host environment to generate and record security events. Following this, system components were integrated to enable automated log ingestion, centralized storage, and visualization.

Database connectivity was implemented to allow persistent storage of collected alerts and logs, ensuring that access events could be retrieved and analyzed efficiently. User interface components were developed to provide administrators with an interactive dashboard capable of displaying access-related information clearly and consistently. Throughout the implementation process, initial functional testing was performed to verify that each component operated correctly both independently and as part of the complete system.

6.3 Implementation Details

6.3.1 User Authentication

User authentication was implemented to restrict access to the Access Tracing administrative dashboard. An authentication mechanism was developed to ensure that only authorized users could view intrusion detection alerts, access logs, and system monitoring information. This mechanism was implemented using server-side session management, where user credentials are validated before granting access to the system interface.

Once authenticated, users are assigned a session that allows continuous access to the dashboard until logout or session expiration. Unauthorized users attempting to access protected pages are redirected to the authentication interface. This implementation ensures that sensitive security information is protected and accessible only to designated administrators, supporting the system's security and operational requirements. Administrative access to the system is restricted through authentication mechanisms, as shown in Figure 6.2.

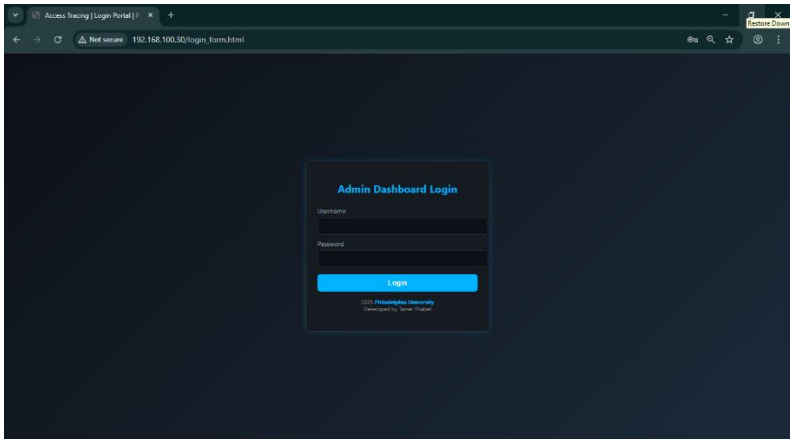

Fig 6.2: Admin Authentication Interface for Access Tracing Dashboard

6.3.2 Database Integration

Database integration was implemented to provide centralized and structured storage for access tracing data. A MySQL database was used to store intrusion detection alerts and host-level records collected from the system. Database tables were designed to capture essential event attributes such as timestamps, alert descriptions, source information, and event types.

The application connects to the database using server-side scripts that handle data insertion and retrieval. When new alerts or logs are ingested, records are inserted automatically into the appropriate tables. Database queries are executed to retrieve recent events, filter results, and support dashboard visualization. This integration ensures data persistence, efficient querying, and reliable access to historical records. Successful database integration is demonstrated in Figure 6.3, which shows intrusion alerts stored and retrieved using SQL queries.

```
Administrator: C:\Windows\system32\cmd.exe - mysql  -u app_user -p
Database changed
mysql> SELECT COUNT(*) FROM snort_alerts;
+----------+
| COUNT(*) |
+----------+
|       22 |
+----------+
1 row in set (0.065 sec)

mysql> SELECT id, event_time, alert_message, protocol, source_ip, destination_ip
    -> FROM snort_alerts
    -> ORDER BY id DESC
    -> LIMIT 30;
+----+----------------------+-----------------------------------------------+----------+---------------+---------
-------+
| id | event_time           | alert_message                                 | protocol | source_ip     | destinat
ion_ip |
+----+----------------------+-----------------------------------------------+----------+---------------+---------
-------+
| 22 | 01/08-16:24:12.717515 | AT WEB TEST: HTTP GET request seen            | TCP      | 192.168.100.20 | 192.168.
100.30 |
| 21 | 01/08-16:24:12.717515 | AT WEB SQLi Attempt (UNION SELECT keyword)    | TCP      | 192.168.100.20 | 192.168.
100.30 |
| 20 | 01/08-16:24:12.717515 | AT WEB SQLi Attempt (sqli.php + encoded quote %27) | TCP      | 192.168.100.20 | 192.168.
100.30 |
| 19 | 01/08-16:24:12.716370 | AT TCP SYN Scan Detected                      | TCP      | 192.168.100.20 | 192.168.
100.30 |
| 18 | 01/08-16:23:54.435426 | AT WEB TEST: HTTP GET request seen            | TCP      | 192.168.100.20 | 192.168.
100.30 |
| 17 | 01/08-16:23:54.435426 | AT WEB SQLi Attempt (sqli.php + encoded quote %27) | TCP      | 192.168.100.20 | 192.168.
```

Fig 6.3: MySQL Database Records Retrieved After Successful Integration

6.3.3 User Interface Elements

The user interface of the Access Tracing system was implemented as a web-based dashboard designed to present access-related information in a clear and organized manner. The dashboard layout includes structured tables and visual components that display intrusion alerts and system events. Interface elements were implemented using standard web technologies to ensure compatibility and responsiveness.

The dashboard dynamically retrieves data from the backend and updates displayed information without requiring manual page reloads. Visual clarity was prioritized to ensure that administrators can quickly identify access attempts, review alert details, and monitor system activity efficiently. The interface design supports usability and accessibility while maintaining a professional appearance suitable for monitoring purposes. The integrated dashboard interface used to visualize access-related events is shown in Figure 6.4.

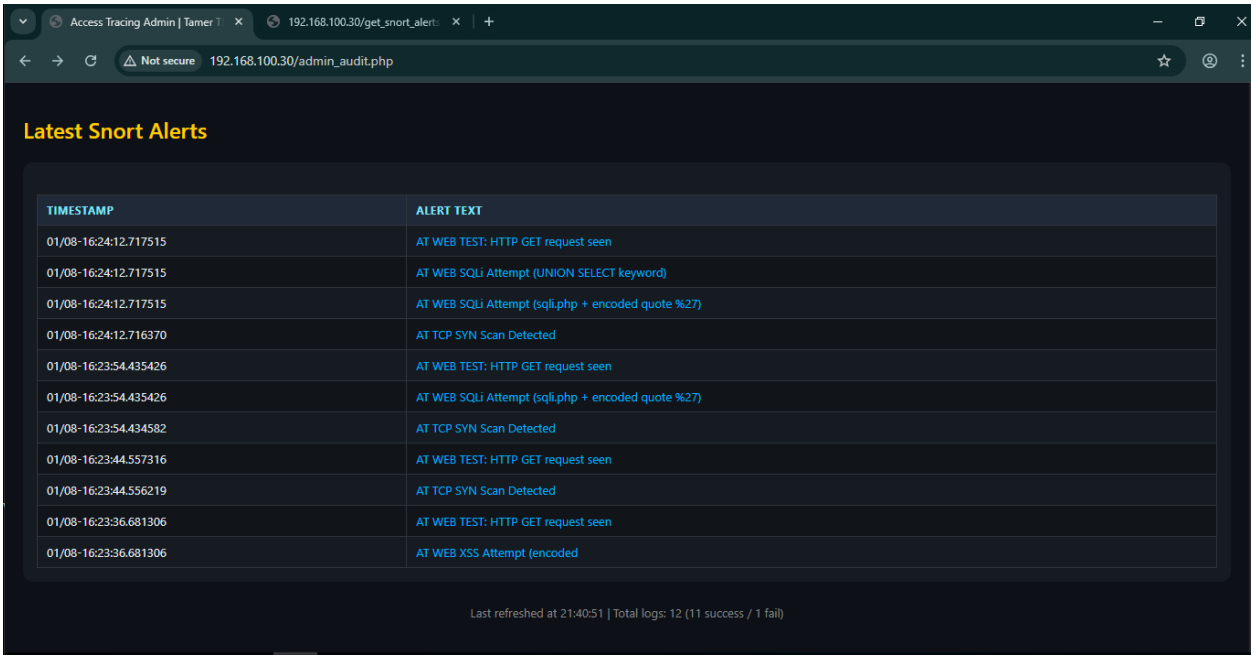

Fig 6.4: Administrative Dashboard User Interface Showing Alert Records

6.3.4 Functionality Modules

The core functionality of the Access Tracing system was implemented through several integrated modules that collectively enable access monitoring and tracing. The intrusion detection module was implemented using Snort IDS to analyze network traffic and generate alerts for suspicious activity. Custom rules were configured to detect reconnaissance activity, SQL Injection attempts, and Cross-Site Scripting payloads.

The log ingestion module was implemented to automatically retrieve intrusion alerts and host-level logs, normalize their structure, and store them in the database. This module ensures that data from different sources is processed consistently and made available for analysis.

Access monitoring functionality was achieved by correlating network alerts with host-level records based on timing and event characteristics. The visualization module displays this information through the administrative dashboard, allowing administrators to observe access attempts as unified events rather than isolated logs. Together, these modules form the operational core of the Access Tracing system.

Correlation between network-level alerts and host-level events is achieved using timestamp proximity and contextual attributes. Intrusion detection alerts generated by Snort include precise timestamps and source IP information. Host-level authentication and application events recorded on the Windows Server also include timestamped records. During ingestion and visualization, events occurring within a defined time window and sharing related network or application context are presented together on the dashboard. The successful execution of the ingestion and correlation workflow is demonstrated in Figure 6.5 Test Scenario Execution Workflow, where detected intrusion events are retrieved from log files, stored in the database, and reflected on the dashboard interface.

```
┌──(dark㉿kali)-[~]
└─$ curl "http://192.168.100.30/sqli.php?id=1%27%20UNION%20SELECT%201,2--%20"
Query:
SELECT id, username FROM users WHERE id = '1' UNION SELECT 1,2-- '

Result:
1  admin
1  2
```

Fig 6.5.1: Attack simulation

```
dark@kali: ~
Session  Actions  Edit  View  Help

┌──(dark㉿kali)-[~]
└─$ sudo snort -i eth1 -c /etc/snort/snort.conf -A fast -q
[sudo] password for dark:
```

Fig 6.5.2: Snort IDS mode detects traffic

```
01/09-16:28:02.084164  [**] [1:4000101:3] AT WEB SQLi Attempt (sqli.php + encoded quote %27) [**]
{TCP} 192.168.100.20:39634 → 192.168.100.30:80
01/09-16:28:02.084164  [**] [1:4000102:3] AT WEB SQLi Attempt (UNION SELECT keyword) [**] [Priorit
2.168.100.20:39634 → 192.168.100.30:80
```

Fig 6.5.3: Alert written to log file

```
PS C:\Users\Administrator> & "C:\Program Files\MySQL\MySQL Server 9.4\bin\mysql.exe" "--user=app_user" "--password=123"
"--database=demo_app" -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+---+
| 1 |
+---+
| 1 |
+---+
PS C:\Users\Administrator> PowerShell.exe -ExecutionPolicy Bypass -File "C:\MyProject\ingest_snort_demo.ps1"
PS C:\Users\Administrator> PowerShell.exe -ExecutionPolicy Bypass -File "C:\MyProject\ingest_snort_demo.ps1"
PS C:\Users\Administrator> notepad C:\MyProject\ingest_snort_demo.ps1
PS C:\Users\Administrator> notepad C:\MyProject\ingest_snort_demo.ps1
PS C:\Users\Administrator> PowerShell.exe -ExecutionPolicy Bypass -File "C:\MyProject\ingest_snort_demo.ps1"
[OK] Lines fetched: 22
[OK] Lines matched (full parse): 22
[OK] Lines fallback-inserted: 0
[OK] Rows inserted (real): 22
[OK] MySQL errors: 0
PS C:\Users\Administrator>
```

Fig 6.5.4: PowerShell ingestion retrieves it, Data inserted into MySQL

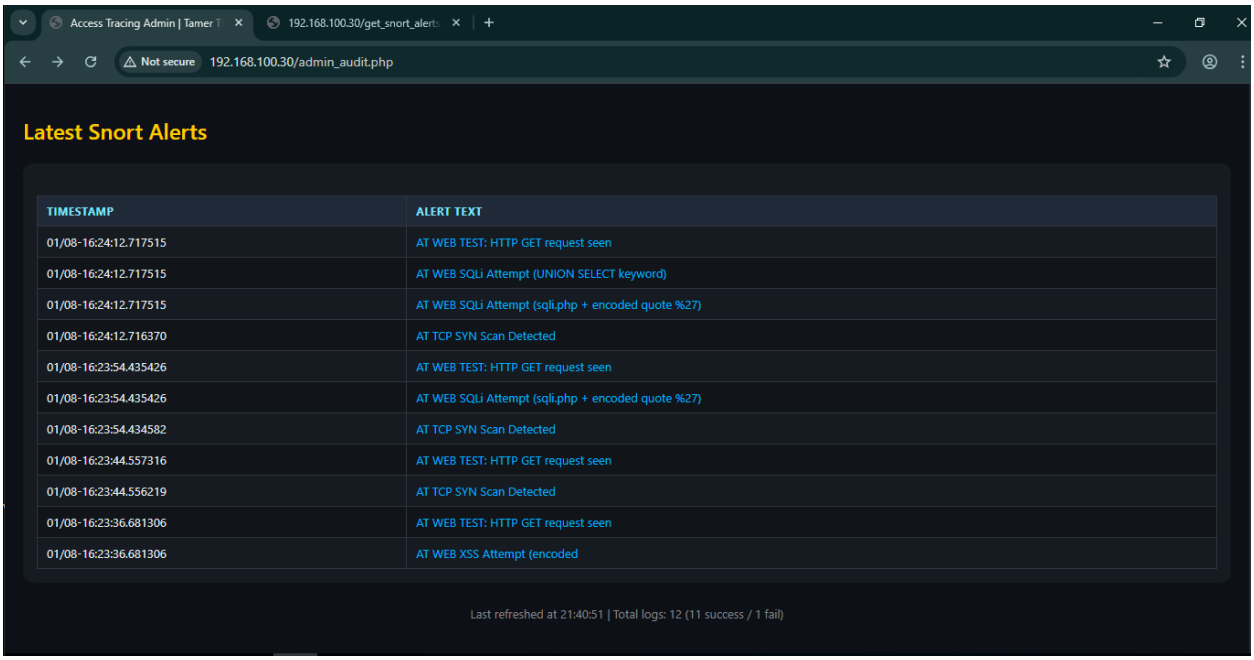

Fig 6.5.5: Dashboard displays it

CHAPTER 7 – RESULTS AND DISCUSSION

7.1 Introduction

This chapter presents the evaluation phase of the Access Tracing project, where the implemented system is examined to determine its effectiveness, reliability, and compliance with the defined requirements. The purpose of this chapter is to analyze the results obtained from system execution and to discuss how these results reflect the system's ability to trace access attempts across network and host layers. Evaluation is conducted through controlled testing scenarios, practical execution, and observation of system behavior during attack simulations. The discussion focuses on whether the system operates as intended and how successfully it fulfills the project objectives.

7.2 Data Collection and Research

The data used for evaluating the Access Tracing system was collected through controlled experiments conducted within a virtual laboratory environment. Rather than relying on external datasets, the system was tested using real-time data generated by deliberate attack scenarios executed against the monitored system. This approach ensured that the collected data accurately reflected system behavior under realistic conditions.

Network-level data was collected in the form of intrusion detection alerts generated by Snort IDS while monitoring traffic directed toward the Windows Server. These alerts included timestamps, source and destination information, and descriptive messages identifying suspicious activity. Host-level data was collected from Windows Server logs, capturing authentication attempts and application-level events resulting from user interactions with the deployed web application.

Additional data was collected from the centralized database, which stored normalized records ingested from both network and host sources. The dashboard output served as a visual representation of the collected data, allowing direct observation of system performance. This combination of network alerts, host logs, database records, and dashboard output formed the basis for evaluating the system's accuracy and traceability.

7.3 System Testing and User Feedback

System testing was conducted to verify that the Access Tracing system met its functional and non-functional requirements. Functional testing focused on validating that each system feature operated correctly according to its specification. This included verifying that Snort IDS successfully detected suspicious traffic, that log ingestion scripts retrieved and processed alerts correctly, that data was stored accurately in the database, and that the dashboard displayed up-to-date access events. Testing confirmed that SQL Injection and Cross-Site Scripting attacks were detected, ingested, and visualized as expected, demonstrating correct system functionality.

Non-functional testing evaluated system performance, usability, and reliability within the constraints of the virtual environment. Performance testing confirmed that alerts were ingested and displayed on the dashboard with minimal delay, supporting near real-time monitoring. Usability testing showed that the dashboard interface was clear and easy to navigate, allowing administrators to review access events efficiently. Reliability testing confirmed that the system continued operating correctly even when log files were temporarily empty or when repeated attack scenarios were executed.

User feedback was collected informally through observation during system demonstration and testing. Feedback indicated that the system provided clear visibility into access attempts and improved understanding of how network and host events relate to each other. Overall, testing results demonstrated that the Access Tracing system is reliable, functional, and effective in achieving unified access monitoring.

CHAPTER 8 – RECOMMENDATIONS AND CONCLUSION

8.1 Introduction

This chapter concludes the Access Tracing project by summarizing the key findings and providing recommendations based on the implementation and evaluation results. The purpose of this chapter is to reflect on the overall project outcome, assess whether the project objectives were achieved, and discuss potential improvements and future enhancements. The conclusions and recommendations presented are based on practical execution, observed system behavior, and identified limitations.

8.2 Conclusion

The Access Tracing project successfully achieved its primary objective of designing and implementing a unified system capable of tracing access attempts across both network and host environments. The system demonstrated the ability to detect suspicious network activity using an intrusion detection mechanism, capture host-level authentication and application events, and correlate these events within a centralized platform. Using a web-based dashboard, access events were presented in a clear and organized manner, enabling effective monitoring and analysis.

The results confirmed that the system addresses the problem of fragmented security visibility by providing a unified view of access behavior. Controlled testing scenarios, including SQL Injection and Cross-Site Scripting attacks, validated the system's ability to trace access attempts from initial network interaction to host-level impact. Overall, the Access Tracing system met its defined objectives and provided a functional solution aligned with the project requirements.

8.3 Limitations and Recommendations for Future Work

Although the Access Tracing system was successfully designed and implemented, several limitations should be acknowledged. The system was developed and evaluated within a controlled virtual laboratory environment, which limits its scalability and does not fully reflect enterprise-scale network conditions. In addition, the testing process relied on simulated attack scenarios, reducing the variety of access behaviors and threat patterns observed. The system primarily focuses on access detection, correlation, and visualization, without implementing automated response mechanisms or advanced analytical techniques.

To enhance the system in future work, the following improvements are recommended:

- Support monitoring of multiple hosts and distributed environments
- Enhance correlation logic to establish deeper relationships between events
- Integrate additional log sources such as application logs or system process logs

- Implement performance optimization for higher event volumes
- Add long-term log retention and historical analysis capabilities
- Introduce automated alerting or response actions for detected threats

REFERENCES

[1] Haas, S., Sommer, R., & Fischer, M. (2020). *Zeek–Osquery host–network correlation framework for security monitoring*. In Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS). IEEE.

[2] Sharma, P., Gupta, R., & Verma, A. (2021). *Unified event analysis platform for host and network monitoring*. International Journal of Information Security and Privacy, 15(2), 1–15.

[3] Albasheer, H., Alharthi, A., & Alqahtani, F. (2022). *Cyber-attack prediction using alert correlation techniques in network intrusion detection systems*. Journal of Information Security and Applications, 64, 103058.

[4] Chen, Y., Wang, L., & Li, X. (2023). *Hybrid host–network intrusion detection using feature fusion*. Computers & Security, 124, 102982.

[5] Maosa, K., Abdelrahman, M., & Hassan, R. (2024). *Hierarchical security event correlation model for real-time attack detection*. IEEE Access, 12, 45678–45692.

APPENDICES:

Export CSV

```
<?php

require __DIR__ . '/auth.php';

if (($_SESSION['role'] ?? '') !== 'admin') { http_response_code(401); exit('Unauthorized'); }


$config = require __DIR__ . '/config.php';

$mysqli = new mysqli($config['db_host'], $config['db_user'], $config['db_pass'],
$config['db_name']);

if ($mysqli->connect_errno) { http_response_code(500); exit('DB error'); }

$mysqli->set_charset('utf8mb4');


$sql = "SELECT id, username, success, ip_address, source, event_id, host, severity,
ti_malicious, created_at

    FROM login_audit ORDER BY id DESC LIMIT 100";

$res = $mysqli->query($sql);


header('Content-Type: text/csv; charset=utf-8');

header('Content-Disposition: attachment; filename=login_audit_last100.csv');


$out = fopen('php://output', 'w');

fputcsv($out,
['id','username','success','ip_address','source','event_id','host','severity','ti_malicious','created_
at']);

if ($res) {

  while ($r = $res->fetch_assoc()) {

    $r['success'] = (int)$r['success'];
```

```
        $r['ti_malicious'] = (int)$r['ti_malicious'];
        fputcsv($out, $r);
    }
    $res->free();
}
fclose($out);
$mysqli->close();
```

Snort version verification:

snort -V

Snort execution in IDS mode:

snort -c /etc/snort/snort.conf -i eth1 -A fast

Snort Rules

```
##############################################
# Access Tracing – Custom Snort Rules
# File: /etc/snort/rules/local.rules
##############################################

# 1. ICMP Ping Detection (Reconnaissance)
alert icmp any any -> $HOME_NET any (
    msg:"ICMP Ping Detected (Reconnaissance)";
    itype:8;
    sid:1000001;
```

```
    rev:1;
)

# 2. TCP SYN Scan Detection (Port Scanning)
alert tcp any any -> $HOME_NET any (
    flags:S;
    msg:"TCP SYN Scan Detected";
    threshold:type threshold, track by_src, count 5, seconds 10;
    sid:1000002;
    rev:1;
)

# 3. SQL Injection – OR 1=1
alert tcp any any -> $HOME_NET 80 (
    msg:"SQL Injection Attempt - OR 1=1";
    flow:to_server,established;
    content:"or 1=1";
    nocase;
    sid:1000003;
    rev:1;
)

# 4. SQL Injection – UNION SELECT
alert tcp any any -> $HOME_NET 80 (
    msg:"SQL Injection Attempt - UNION SELECT";
    flow:to_server,established;
```

```
    content:"union select";
    nocase;
    sid:1000004;
    rev:1;
)

# 5. SQL Injection – Quote Injection
alert tcp any any -> $HOME_NET 80 (
    msg:"SQL Injection Attempt - Quote Injection";
    flow:to_server,established;
    content:"'";
    http_uri;
    sid:1000005;
    rev:1;
)

# 6. XSS – Script Tag
alert tcp any any -> $HOME_NET 80 (
    msg:"XSS Attempt - Script Tag";
    flow:to_server,established;
    content:"<script>";
    nocase;
    sid:1000006;
    rev:1;
)
```

```
# 7. XSS – JavaScript Alert
alert tcp any any -> $HOME_NET 80 (
   msg:"XSS Attempt - JavaScript Alert";
   flow:to_server,established;
   content:"alert(";
   nocase;
   sid:1000007;
   rev:1;
)

# 8. XSS – IMG Onerror Payload
alert tcp any any -> $HOME_NET 80 (
   msg:"XSS Attempt - IMG Onerror Payload";
   flow:to_server,established;
   content:"onerror=";
   nocase;
   sid:1000008;
   rev:1;
)
```

Authentication

```
<?php
if (session_status() === PHP_SESSION_NONE) {
   $sessPath = 'C:\\MyProject\\sessions';
   if (!is_dir($sessPath)) {
      mkdir($sessPath, 0777, true);
   }
```

```
    ini_set('session.save_path', $sessPath);
    session_start();
}

if (!isset($_SESSION['username'])) {
    http_response_code(401);
    echo "<div style='
        font-family:Segoe UI,Arial;
        background:#0d1117;
        color:#ff5555;
        text-align:center;
        padding:60px;
        font-size:18px;'>
        <h2>Unauthorized Access</h2>
        <p>You must log in to view this page.</p>
        <a href='/login_form.html' style='color:#00b3ff;'>Return to Login</a>
    </div>";
    exit;
}

if (isset($require_admin) && $require_admin === true) {
    if (($_SESSION['role'] ?? '') !== 'admin') {
        http_response_code(403);
        echo "<div style='
            font-family:Segoe UI,Arial;
            background:#0d1117;
```

```
        color:#ffcc00;
        text-align:center;
        padding:60px;
        font-size:18px;'>
        <h2>Access Denied</h2>
        <p>This section is for administrators only.</p>
        <a href='/login_form.html' style='color:#00b3ff;'>Return to Login</a>
    </div>";
    exit;
  }
}
?>
```

Get Snort Alerts

```
<?php

header('Content-Type: application/json; charset=utf-8');
header('Cache-Control: no-cache, must-revalidate');
error_reporting(0);

// 1) Where to read Snort alerts from (Kali CSV)
$csvUrl = 'http://192.168.100.20/snort_alerts.csv';

// 2) How many alerts max to send to the dashboard
$MAX_ALERTS = 100;
```

```
// 3) Load CSV from Kali
$data = @file_get_contents($csvUrl);
if ($data === false || trim($data) === '') {
    echo json_encode(["error" => "Failed to load CSV"], JSON_PRETTY_PRINT);
    exit;
}

// 4) Split into lines
$lines = preg_split('/\r\n|\n|\r/', trim($data));

// 5) Walk from bottom (newest) ? up, dedupe & limit
$rows = [];
$seen = [];

for ($i = count($lines) - 1; $i >= 0; $i--) {
    $line = trim($lines[$i]);
    if ($line === '') continue;

    // skip header or junk
    if (stripos($line, 'Timestamp') === 0) continue;
    if (stripos($line, 'status') === 0) continue;

    $parts = str_getcsv($line);
    if (count($parts) < 2) continue;

    $ts   = trim($parts[0]);
```

```
    $text = trim($parts[1]);

    if ($text === '' || strlen($text) < 5) continue;

    // 6) Dedup key: same second + same message = one alert
    // Example ts: "11/20-09:32:10.801847" ? "11/20-09:32:10"
    $tsShort = substr($ts, 0, 17);
    $key = $tsShort . '|' . $text;

    if (isset($seen[$key])) {
        continue; // already counted this "type" of alert at that second
    }
    $seen[$key] = true;

    $rows[] = [
        'Timestamp' => $ts,
        'AlertText' => $text
    ];

    if (count($rows) >= $MAX_ALERTS) {
        break;
    }
}

// 7) Put back into chronological order (oldest ? newest)
$rows = array_reverse($rows);
```

```
// 8) Output clean JSON array
if (ob_get_length()) {
    ob_clean();
}
echo json_encode($rows, JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
```

Example SQL query for stored alerts

```
SELECT * FROM snort_alerts ORDER BY event_time DESC;
```

PowerShell log ingestion execution

```
PowerShell.exe -ExecutionPolicy Bypass -File ingest_logs.ps1
```

Login Form

```
session_save_path('C:\\MyProject\\sessions');
$config = require __DIR__ . '/config.php';

// --- Connect to Database ---
$mysqli = new mysqli($config['db_host'], $config['db_user'], $config['db_pass'], $config['db_name']);
if ($mysqli->connect_errno) {
    error_log("DB connect error: " . $mysqli->connect_error);
    http_response_code(500);
    echo "Database connection failed.";
    exit;
```

```
}

// --- Ensure login_audit table exists ---
$mysqli->query("
CREATE TABLE IF NOT EXISTS login_audit (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100),
    success TINYINT(1),
    ip_address VARCHAR(45),
    user_agent TEXT,
    source VARCHAR(20) DEFAULT 'web',
    event_id VARCHAR(32) DEFAULT NULL,
    host VARCHAR(100) DEFAULT NULL,
    log_type VARCHAR(50) DEFAULT NULL,
    severity VARCHAR(20) DEFAULT NULL,
    ti_malicious TINYINT(1) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
");

// --- Start session ---
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// --- Collect Data ---
$username = trim($_POST['username'] ?? '');
```

```
$password = $_POST['password'] ?? '';

// Fix: Standardize IPv6 loopback (::1) to IPv4 loopback (127.0.0.1)
$ip = $_SERVER['REMOTE_ADDR'] ?? 'unknown';
if ($ip === '::1' || $ip === ':1') { // Check for both common IPv6 loopbacks
    $ip = '127.0.0.1';
}

$ua = $_SERVER['HTTP_USER_AGENT'] ?? 'unknown';
$source = 'web';
$success = 0;
// ... rest of code
// --- Validate Input ---
if ($username === '' || $password === '') {
    echo "Please enter username and password.";
    exit;
}

// --- Check for lockout (too many fails) ---
$lockCheck = $mysqli->prepare("
    SELECT COUNT(*) FROM login_audit
    WHERE username=? AND success=0 AND created_at > NOW() - INTERVAL 1 MINUTE
");
$lockCheck->bind_param('s', $username);
$lockCheck->execute();
$lockCheck->bind_result($failCount);
$lockCheck->fetch();
```

```
$lockCheck->close();

if ($failCount >= 5) {
   echo "Account temporarily locked due to multiple failed attempts. Try again in 1 minute.";
    $stmt = $mysqli->prepare("INSERT INTO login_audit (username, success, ip_address, user_agent, source, severity) VALUES (?, 0, ?, ?, 'web', 'warning')");
   $stmt->bind_param('sss', $username, $ip, $ua);
   $stmt->execute();
   $stmt->close();
   exit;
}

// --- Verify user credentials ---
$stmt = $mysqli->prepare("SELECT id, username, password_hash, display_name, role FROM users WHERE username=? LIMIT 1");
$stmt->bind_param('s', $username);
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_assoc();
$stmt->close();

if ($user && password_verify($password, $user['password_hash'])) {
   $success = 1;
   $_SESSION['user_id'] = $user['id'];
   $_SESSION['username'] = $user['username'];
   $_SESSION['display_name'] = $user['display_name'];
   $_SESSION['role'] = $user['role'];
   $severity = 'info';
```

```
    echo "? Login successful! Welcome, " . htmlspecialchars($user['display_name'] ?? $user['username']);
} else {
   $success = 0;
   $severity = 'high';
   echo "? Login failed.";
}

// --- Insert into audit log ---
$stmt = $mysqli->prepare("
INSERT INTO login_audit (username, success, ip_address, user_agent, source, severity)
VALUES (?, ?, ?, ?, ?, ?)
");
$stmt->bind_param('sissss', $username, $success, $ip, $ua, $source, $severity);
$stmt->execute();
$stmt->close();

$mysqli->close();

// --- Redirect on success ---
if ($success === 1) {
   header("Refresh: 1; URL=admin_audit.php");
   exit;
}
?>
```

API

```
<?php-

$config = require __DIR__ . '/config.php';


$API_SECRET =
'c6e9f1a2b0d3e5f8a7b9c0d1e2f3a4b5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0'; // <-- KEY


if (($_SERVER['HTTP_X_API_KEY'] ?? '') !== $API_SECRET) {

   http_response_code(401);

   die('Unauthorized API Key.');

}

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {

   http_response_code(405);

   die('Method Not Allowed.');

}


$mysqli = new mysqli($config['db_host'], $config['db_user'], $config['db_pass'],
$config['db_name']);

if ($mysqli->connect_errno) {

   http_response_code(500);

   error_log("API DB connect error: " . $mysqli->connect_error);

   die("Database error.");

}


$json_data = file_get_contents('php://input');

$log_data = json_decode($json_data, true);
```

```
if (empty($log_data) || !is_array($log_data)) {
   http_response_code(400);
   die('Invalid JSON input.');
}

$username   = $log_data['username'] ?? 'SYSTEM_EVENT';
$success    = $log_data['success'] ?? 0;
$ip_address = $log_data['ip_address'] ?? '127.0.0.1';
$source     = $log_data['source'] ?? 'system';
$event_id   = $log_data['event_id'] ?? NULL;
$host       = $log_data['host'] ?? 'WindowsServer';
$severity   = $log_data['severity'] ?? 'info';
$log_type   = $log_data['log_type'] ?? 'security';
$user_agent = $log_data['user_agent'] ?? 'PowerShell-Script';

$stmt = $mysqli->prepare("
   INSERT INTO login_audit (username, success, ip_address, user_agent, source, event_id,
host, log_type, severity)
   VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
");
$stmt->bind_param('sisssisss', $username, $success, $ip_address, $user_agent, $source,
$event_id, $host, $log_type, $severity);

if ($stmt->execute()) {
   http_response_code(200);
   echo "Log entry created successfully.";
} else {
```

```
    http_response_code(500);
    error_log("DB execution error: " . $stmt->error);
    echo "Log insertion failed.";
}

$stmt->close();
$mysqli->close();
?>
```

Logout

```
<?php
session_start();
$_SESSION = [];
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}
session_destroy();
header('Location: /login_form.html');
exit;
```